

INFERCEPT: Efficient Intercept Support for Augmented Large Language Model Inference

Reyna Abhyankar^{*1} Zijian He^{*1} Vikranth Srivatsa¹ Hao Zhang¹ Yiyi Zhang¹

Abstract

Large language models are increasingly integrated with external environments, tools, and agents like ChatGPT plugins to extend their capability beyond language-centric tasks. However, today’s LLM inference systems are designed for standalone LLMs. They treat each external interaction as the end of LLM generation and form a new request when the interaction finishes, causing unnecessary recomputation of already computed contexts, which accounts for 37-40% of total model forwarding time. This paper presents INFERCEPT, the first LLM inference framework targeting augmented LLMs and supporting the efficient interception of LLM generation. INFERCEPT minimizes the GPU resource waste caused by LLM interceptions and dedicates saved memory for serving more requests. INFERCEPT improves the overall serving throughput by $1.6\times$ - $2\times$ and completes $2\times$ more requests per second compared to the state-of-the-art LLM inference systems.

1 Introduction

Large language models (LLMs) (Achiam et al., 2023; Touvron et al., 2023) have shown immense potential in various applications, such as natural language understanding and content generation. Nonetheless, LLMs alone can only generate text. To extend LLMs’ capabilities to handle more diverse and open-ended tasks, a recent trend is to augment LLMs with external tools (Mialon et al., 2023; Wang et al., 2023; Qin et al., 2023) such as arithmetic calculation (Hao et al., 2023) and ChatGPT plugins (OpenAI, 2023), non-LLM models like image generation (Betker et al., 2024),

and interaction with humans and other environments in real-time, such as virtual environments (Shridhar et al., 2021). Notably, Ng poses that “AI agent workflows will drive massive AI progress in 2024” (Ng, 2024). More generally, we can view the use of tools and the interaction with humans or other environments all as LLMs being augmented and *intercepted* by external entities.

Augmentations have unique properties that bring new problems to LLM inference systems. First, they *intercept* regular LLM decoding, *i.e.*, when an external tool is called or when waiting for human/environment responses, the normal decoding phase is paused and can only resume when the interception finishes. Second, an interception’s time varies drastically across augmentation types and requests. For example, a calculator API finishes in less than 1 ms, yet it may take humans more than 1 minute to digest and type a subsequent chat prompt. This complicates request scheduling. Third, unlike regular requests, the context (*i.e.*, KV caches) cannot be used for a paused request during interceptions but will be needed upon the end of interceptions. It is unclear how to handle *temporarily unused context*. Adding to the complexity is the high variation in context length and interception time across requests.

Unfortunately, existing LLM inference systems (Kwon et al., 2023; Yu et al., 2022; Li et al., 2023; Aminabadi et al., 2022; NVIDIA, 2023; Agrawal et al., 2023) fall short in serving augmented LLMs with interceptions. They typically interpret an interception as a termination signal – they conclude the ongoing request when the interception happens, discard its context, relegate the task to the augmenting entity, and subsequently re-initiate a new request upon receiving the response from the entity. Consequently, when a request frequently triggers interceptions, for each interception, the initial prompt and all tokens generated before the interception are formed as a new request submitted to the inference system, causing substantial recomputation of keys and values (known as the KV cache) for all previously processed tokens. This recomputation wastes GPU resources, which could instead serve new requests.

In contrast to the aforementioned interception handling approach (which we refer to as Discard), an alternative approach is to preserve a request’s context during the inter-

^{*}Equal contribution ¹University of California, San Diego, La Jolla, United States. Correspondence to: Reyna Abhyankar <vabhyank@ucsd.edu>, Zijian He <zih015@ucsd.edu>, Yiyi Zhang <yiyi@ucsd.edu>.

INFERCEPT: 面向增强型大语言模型推理的高效拦截支持

Reyna Abhyankar^{*1} Zijian He^{*1} Vikranth Srivatsa¹ Hao Zhang¹ Yiyi Zhang¹

摘要

大语言模型正日益与外部环境、工具及代理（如ChatGPT插件）集成，以扩展其超越语言中心任务的能力。然而，当今的LLM推理系统是为独立LLM设计的。它们将每次外部交互视为大语言模型生成的终点，并在交互结束时形成新的请求，导致对已计算上下文的非必要重计算，这占用了模型前向传播总时间的37%至40%。本文提出INFERCEPT，这是首个面向增强型LLM的推理框架，支持对LLM生成过程的高效拦截。INFERCEPT最大程度减少因LLM拦截导致的GPU资源浪费，并将节省的内存用于服务更多请求。与最先进的LLM推理系统相比，INFERCEPT将整体服务吞吐量提升 $1.6\times$ - $2\times$ 倍，每秒处理请求数增加 $2\times$ 个。

1 引言

大型语言模型 (LLMs) (Achiam et al., 2023; Touvron et al., 2023) 在各种应用中展现出巨大潜力，例如自然语言理解和内容生成。尽管如此，仅靠LLMs本身只能生成文本。为了扩展LLMs的能力以处理更多样化和开放式任务，最近的一种趋势是利用外部工具增强LLMs (Mialon et al., 2023; Wang et al., 2023; Qin et al., 2023) 例如算术计算 (Hao et al., 2023) 和ChatGPT插件 (OpenAI, 2023)，非大语言模型（如图像生成）(Betker et al., 2024)，

^{*}同等贡献 ¹加利福尼亚大学圣迭戈分校，美国拉霍亚。通讯作者: Reyna Abhyankar<vabhyank@ucsd.edu>, Zijian He <zih015@ucsd.edu>, Yiyi Zhang <yiyi@ucsd.edu>。

以及与人和其他环境的实时交互，例如虚拟环境 (Shridhar et al., 2021年)。值得注意的是，Ng提出“AI代理工作流将在2024年推动巨大的AI进展” (Ng, 2024)。更一般地说，我们可以将工具的使用以及与人或其他环境的交互都视为LLMs被增强和拦截由外部实体。

增强具有独特的属性，给大语言模型推理系统带来了新的问题。首先，它们拦截常规的大语言模型解码，即，当调用外部工具或等待人类/环境响应时，正常的解码阶段被暂停，只能等待拦截结束后恢复。其次，不同增强类型和请求的拦截时间差异巨大。例如，计算器API在不到1毫秒内完成，而人类可能需要超过1分钟来消化并输入后续聊天提示。这使得请求调度变得复杂。第三，与常规请求不同，暂停请求的上下文（即，KV缓存）在拦截期间无法使用，但在拦截结束时需要用到。如何处理临时未使用的上下文尚不清楚。更复杂的是，不同请求的上下文长度和拦截时间差异很大。

遗憾的是，现有的大语言模型推理系统 (Kwon et al., 2023; Yuet et al., 2022; Li et al., 2023; Aminabadi et al., 2022; 英伟达, 2023; Agrawal et al., 2023) 在支持带拦截的增强型大语言模型方面表现不足。它们通常将拦截视为一种终止信号——当拦截发生时，它们会结束当前请求，丢弃其上下文，将任务移交给增强实体，并在收到该实体的响应后重新发起一个新请求。因此，当一个请求频繁触发拦截时，对于每次拦截，初始提示和所有在拦截之前生成的标记都会作为新请求提交给推理系统，导致对之前所有已处理标记的键和值（即KV缓存）进行大量重计算。这种重计算浪费了原本可以用于服务新请求的GPU资源。

与前述的拦截处理方法（我们称之为“丢弃”）不同，另一种方法是在拦截期间保留请求的上下文，并在inter-

ception and resume the request when the interception finishes. This approach, which we call *Preserve*, avoids recomputation but occupies GPU memory during the entire interception. This GPU memory could accommodate other requests, which increases throughput (Kwon et al., 2023).

To avoid recomputation and memory preservation, another possible approach to handle interceptions is to swap contexts to CPU memory when an interception happens (we call it *Swap*). Although *Swap* avoids recomputation and GPU memory wastage, with limited GPU-CPU link bandwidth, foreground tasks (normal forwarding) could be bottlenecked by waiting for swapping to finish. The GPU memory could be used for processing more requests.

How should an LLM inference system efficiently support interceptions? To answer this question, we first study the computational patterns of six typical augmentations: arithmetic, question-and-answer, virtual environment, chatbot, image generation, and text-to-speech transformation. We then evaluate how well the state-of-the-art LLM inference system, vLLM (Kwon et al., 2023) (which performs *Discard*), and the two possible extensions (*Preserve* and *Swap*) perform on these augmentations.

Based on our findings, we propose INFERCEPT, the *first* LLM inference framework targeting augmented LLMs with interceptions. The core idea of INFERCEPT, which we call *min-waste interception*, is to minimize GPU memory waste caused by interceptions so that the same amount of GPU resources can be used to serve more requests. To this end, INFERCEPT incorporates three contributions. First, we deduce three waste-calculation equations to quantify the GPU memory waste of *Discard*, *Preserve*, and *Swap*.

Second, we improve individual interception techniques to reduce or eliminate their memory waste. For *Swap*, we propose swap pipelining, which overlaps swap and foreground computation in a model-layer-by-layer manner. Additionally, we split the swapping of a sequence into multiple model-forwarding iterations so that each iteration’s swapping needs stays within what the GPU-CPU link can sustain (*i.e.*, a swap budget). As a result, INFERCEPT completely eliminates GPU memory waste of *Swap*. For *Discard*, we observe that certain GPU processing capacity is unused by running requests at each iteration because of decoding’s imbalanced memory/compute requirements. We utilize this unused capacity for recomputation. We split requests’ contexts into chunks, each small enough to fit in the unused capacity and recomputed in one iteration.

Finally, INFERCEPT dynamically chooses the interception and resumption strategies within and across requests to minimize overall GPU memory waste while ensuring fairness. For interception scheduling, we sort intercepted requests by their potential memory waste and assign the swap

budget to requests with the highest potential waste. We discard or preserve the remaining requests’ context based on the waste comparison. For resumption scheduling, we assign the swap budget to swapped-out requests with the earliest original arrival times. We also schedule discarded requests, preserved requests, and non-intercepted waiting requests according to FCFS (first-come first-served), adding requests until the GPU’s processing capacity is reached.

We implement INFERCEPT on top of vLLM (Kwon et al., 2023), a state-of-the-art LLM inference system. We evaluate INFERCEPT, vLLM (*Discard*), *Preserve*, and *Swap* on A100 GPUs using three LLMs (GPT-J-6B (Wang & Komatsuzaki, 2021), Vicuña-13B (Zheng et al., 2023a), and Llama3-70B (Meta, 2024)) and the six interception types we study. Overall, INFERCEPT sustains $1.6 \times 2 \times$ higher serving load than vLLM while maintaining similar latency per token generation. INFERCEPT also achieves over $2 \times$ more completed requests per second.

INFERCEPT is available at <https://github.com/WukLab/InferCept>.

2 Augmented LLMs

This section first discusses popular augmented LLM frameworks and then presents properties of typical augments from our empirical study.

2.1 Augmented LLM Frameworks

To extend LLMs’ ability to undertake more types of tasks, different approaches have been proposed to *augment* an LLM with various external entities (Mialon et al., 2023). Below, we describe typical augmentations in three categories.

The first type of augmentation involves non-LLM tools called during the decoding phase of an LLM, usually aimed at extending the types of tasks the LLM can handle. These tools range from simple tools like calculator (Wolfram, 2023) and calendar to real-world interactions like information retrieval (Baeza-Yates et al., 1999) and restaurant reservation (OpenTable, 2023). A tool augmentation can also be another machine-learning model such as translation (Costa-jussà et al., 2022), and question-answering (QA) (Izacard et al., 2022). Previous research (Parisi et al., 2022; Hao et al., 2023; Schick et al., 2023) has shown the effectiveness of fine-tuning an LLM to generate appropriate tool calls automatically during the decoding phase. Instead of fine-tuning, another approach is instructing LLMs to select tools or models for a user task (Shen et al., 2023; Lu et al., 2023).

The second type augments a single triggering of an LLM by making a *sequence* of calls to the same LLM. For exam-

ception并恢复请求，当拦截结束时。这种方法，我们称之为 *Preserve*，避免了重计算，但在整个拦截期间占用了GPU内存。这部分GPU内存本可用于其他请求，从而增加吞吐量 (Kwonetal., 2023年)。

为了避免重计算和内存保留，另一种处理拦截的可能方法是在拦截发生时将上下文换入CPU内存（我们称之为 *Swap*）。虽然`Swap`避免了重计算和GPU内存浪费，但由于GPU-CPU链路带宽有限，前台任务（正常转发）可能会因等待换入完成而成为瓶颈。GPU内存可用于处理更多请求。

*LLM推理系统应如何高效支持拦截？*为回答此问题，我们首先研究了六种典型增强的计算模式：算术、问答、虚拟环境、聊天机器人、图像生成和文本到语音转换。然后，我们评估了最先进的LLM推理系统vLLM (Kwonetal.,2023年)（执行丢弃策略）以及两种可能的扩展（*Preserve*和`Swap`）在这些增强场景上的表现。

基于我们的发现，我们提出了INFERCEPT，这是首个针对带有拦截功能的增强型LLM的LLM推理框架。INFERCEPT的核心思想，我们称之为“最小浪费拦截”，旨在最大限度地减少由拦截引起的GPU内存浪费，从而使用相同数量的GPU资源来服务更多请求。为此，INFERCEPT包含三项贡献。首先，我们推导了三个浪费计算公式，用于量化丢弃、保留和交换的GPU内存浪费。

其次，我们改进了单个拦截技术以减少或消除其内存浪费。对于交换，我们提出了交换流水线，它以逐模型层的方式重叠交换和前台计算。此外，我们将一个序列的交换拆分为多个模型前向传播迭代，使得每次迭代的交换需求保持在GPU-CPU链路所能承受的范围内（即交换预算）。因此，INFERCEPT完全消除了交换的GPU内存浪费。对于丢弃，我们观察到，由于解码的内存/计算需求不平衡，每次迭代中正在运行的请求会留下部分未使用的GPU处理能力。我们利用这种未使用的能力进行重新计算。我们将请求的上下文拆分为多个块，每个块小到足以适应未使用的容量，并在一次迭代中重新计算。

最后，`InferCept`动态地在请求内部及跨请求之间选择拦截和恢复策略，以最小化整体GPU内存浪费，同时确保公平性。对于拦截调度，我们根据潜在内存浪费对拦截的请求进行排序，并将交换

预算分配给潜在浪费最高的请求。我们根据浪费比较来丢弃或`Preserve`其余请求的上下文。对于恢复调度，我们将交换预算分配给具有最早原始到达时间的已交换出的请求。我们还按照先来先服务（FCFS）调度已丢弃的请求、`Preserve`的请求以及未拦截的等待请求，添加请求直至达到GPU的处理能力。

我们在vLLM (Kwonet al., 2023年)这一先进的大语言模型推理系统基础上实现了INFERCEPT。我们使用三个大语言模型（GPT-J-6B (Wang&Komatsuzaki, 2021年)、Vicu`na-13B (Zheng et al., 2023a年)和Llama3-70B (Meta, 2024) )以及我们研究的六种拦截类型，在A100 GPU上评估了INFERCEPT、vLLM（丢弃）、`Preserve`和`Swap`。总体而言，INFERCEPT在保持相似每令牌生成延迟的同时，能承受的服务负载比vLLM高 $1.6 \times 2 \times$ 倍。此外，INFERCEPT每秒完成的请求数还超过了 $2 \times \%$ 。

INFERCEPT可在<https://github.com/WukLab/InferCept>.

2 增强型LLM

本节首先讨论流行的增强型LLM框架，然后根据我们的实证研究介绍典型增强的特性。

2.1 增强的大语言模型框架

为了扩展大语言模型的能力以承担更多类型的任务，已有多种方法被提出，用于增强大语言模型与各种外部实体 (Mialon等人, 2023年)。下面，我们描述三类别中的典型增强方式。

第一种增强类型涉及在大语言模型解码阶段调用的非LLM工具，通常旨在扩展大语言模型能够处理的任务类型。这些工具涵盖从简单的工具如计算器（Wolfram, 2023年）和日历，到现实世界交互如信息检索 (Baeza-Yates等人, 1999) 和餐厅预订（OpenTable, 2023年）。工具增强也可以是另一个机器学习模型，例如翻译（Costa-juss`a等人, 2022）和问答（QA）（Izacard等人, 2022）。先前的研究（Parisi等人, 2022；Hao等人, 2023年；Schick等人, 2023年）已证明在解码阶段微调大语言模型以自动生成适当工具调用的有效性。另一种方法是指令式LLM来选择用户任务的工具或模型（Shen等人, 2023年；Lu等人, 2023年），而不是微调。

第二种类型通过对同一大语言模型进行序列的调用来增强单次触发。例如-

Type	Int Time (sec)	Num Interceptions	Context Len
Math	(9e-5, 6e-5)	(3.75, 1.3)	(1422, 738)
QA	(0.69, 0.17)	(2.52, 1.73)	(1846, 428)
VE	(0.09, 0.014)	(28.18, 15.2)	(2185, 115)
Chatbot	(28.6, 15.6)*	(4.45, 1.96)	(753, 703)
Image	(20.03, 7.8)†	(6.91, 3.93)*	(1247, 792)
TTS	(17.24, 7.6)†	(6.91, 3.93)*	(1251, 792)

Table 1: **Interception Properties** Each cell shows (mean, variance) of the augment. * estimated behavior. † partially estimated.

ple, to continuously interact with a human, a chatbot can take rounds of back-and-forth chats by calling an LLM at each step and maintaining a chat history (Greyling, 2023). Another popular use case is decomposing a complex task into multiple steps. For example, Chain-of-Thought (Wei et al., 2023; Zhang et al., 2023) breaks one request into a chain of “thought” steps, each being a new prompt to an LLM that continues the history of thoughts. Similarly, Re-Act (Yao et al., 2023) uses a chain of reasoning and action steps to accomplish complex tasks.

The third type allows for more complex compositions of LLMs, other models, and tools. Frameworks such as LangChain (Chase, 2022), DSpy (Khattab et al., 2024), Gorilla (Patil et al., 2023), SGLang (Zheng et al., 2023b), and AgentGraph (Chen et al., 2019) provide programming models for users to write their own flows of augmented LLMs. Other efforts enable LLMs to generate compositions of augmented LLMs (Suris et al., 2023; Qian et al., 2023).

2.2 Augmenting Entities and Their Properties

To understand augmented LLM interceptions and design inference systems for them, we first examine a set of representative augmentations, including arithmetic (math), question-and-answer (QA), virtual environment (VE), chatbot, image generation, and text-to-speech (TTS), to answer three key questions: 1) how long does an interception take? 2) how many interceptions occur for a request? and 3) what is the context length when an interception happens? Table 1 summarizes our results with averages and variations. We leave other metrics, such as interception returned token length, detailed CDF distributions, and our evaluation methodology, in the Appendix.

Arithmetic (Math). To solve complex math problems, an LLM can be augmented to deconstruct the problem and call a calculator tool step-by-step. We evaluate this use case with the GSM8K-XL (Hao et al., 2023) dataset, which contains 8.5K high-quality grade-school math problems. As expected, the calculator’s execution times are short, an average of 0.2 ms. It has a fairly large context length when calculators are called since we must prompt the LLM with demonstrations of solving problems step-by-step.

Knowledge-based question and answer (QA). To allow

pre-trained LLMs to access wider sets of knowledge, one can augment the LLM to call knowledge-based QA tools that retrieve information from a rich dataset. We use the Multihop QA Wikipedia (Yang et al., 2018) dataset to evaluate this use case. While having longer context lengths for questions, these QA tools provide relatively quick yet variable execution times due to the non-deterministic network communication latency with Wikipedia.

Virtual Environment (VE). An LLM can be augmented to interact with virtual environments and accomplish complex tasks by performing gradual steps (Gu et al., 2022; Huang et al., 2022; Hu et al., 2023). To evaluate VE, we use the ALFWorld dataset (Shridhar et al., 2021), an interactive environment that aligns text descriptions and commands with a physically embodied robotic simulation. The VE interaction has a short and stable interception time due to a locally executed and embodied text-based environment. The context length is long due to the instructions needed to prompt the LLM to understand VE action sequences.

Chatbot. A popular task for LLMs is a chatbot, which interacts with humans with a sequence of chats. When a human receives a chat response from the LLM, the LLM generation sequence is essentially intercepted. The LLM generation is resumed when the human sends the subsequent chat message. During this process, the chat history must be kept as the context for the entire chat sequence. We use the ShareGPT dataset (Zheng et al., 2023a), which contains crowd-sourced real-user ChatGPT conversations to evaluate chatbots. We estimate human response time (*i.e.*, the interception time) as the summation of human scanning time (Bryshaert, 2019) of the previous chat response and typing time (Ma et al., 2015) of the next prompt. The context lengths vary greatly due to the variety of human prompts. Its interception time also has a high variance.

Image generation. An LLM can be augmented to generate an image or a sequence of images when a human gradually refines features through multiple prompts (*e.g.*, adding details to a face depiction). This involves two interceptions: one for triggering an image-generation model and another for receiving user subsequent prompts. To understand this use case, we use ChatGPT to create a dataset by generating a series of image-generation prompts, each triggering a call to the Stable Diffusion model (Rombach et al., 2021). This workload’s interception time has high variation due to variations in executing the diffusion model and the variation in human response time. The average context length is shorter than the chatbot interception due to smaller input prompts to the diffusion model.

Text-To-Speech (TTS). As demonstrated by the ChatGPT Whisper API support (Brockman et al., 2023), TTS can be integrated with an LLM to communicate with a human. Similar to our image-generation dataset, we use Chat-

Type	拦截时间（秒）	拦截次数	上下文长度
Math	(9e-5, 6e-5)	(3.75, 1.3)	(1422, 738)
QA	(0.69, 0.17)	(2.52, 1.73)	(1846, 428)
VE	(0.09, 0.014)	(28.18, 15.2)	(2185, 115)
聊天机器人	(28.6, 15.6)*	(4.45, 1.96)	(753, 703)
图像	(20.03, 7.8)†	(6.91, 3.93)*	(1247, 792)
TTS	(17.24, 7.6)†	(6.91, 3.93)*	(1251, 792)

表1: **拦截属性**每个单元格显示增强的（均值，方差）。* 估计的行为。† 部分估计。

例，以持续与人类交互，聊天机器人可以轮流进行来回聊天，通过在每一步调用大语言模型并维护聊天历史（Greyling, 2023年）。另一个流行的用例是将复杂任务分解为多个步骤。例如，思维链（Wei等人, 2023年; Zhang等人, 2023年）将一个请求分解为一系列“思考”步骤，每个步骤都是向大语言模型的新提示，延续思考历史。类似地，Re-Act（Yao等人, 2023年）使用推理和行动步骤链来完成复杂任务。

第三种类型允许更复杂的组合，包括大语言模型、其他模型和工具。诸如LangChain (Chase, 2022)、DSpy (Khattabet al., 2024)、Gorilla (Patil et al., 2023年)、SGLang (Zheng et al., 2023b) 和AgentGraph (Chen et al., 2019年) 等框架为用户提供了编写自己增强型大语言模型流程的编程模型。其他工作使得大语言模型能够生成增强型大语言模型的组合 (Suris et al., 2023年; Qian et al., 2023年)。

2.2 增强实体及其属性

为了理解增强型大语言模型拦截并为其设计推理系统，我们首先考察一组代表性增强，包括算术（数学）、问答（QA）、虚拟环境（VE）、聊天机器人、图像生成和文本转语音（TTS），以回答三个关键问题：1) 一次拦截需要多长时间？2) 一个请求会发生多少次拦截？3) 拦截发生时上下文长度是多少？表1总结了我们的结果，包含均值和变化。我们将其他指标，如拦截返回令牌牌长度、详细的累积分布函数分布和我们的评估方法，留在了附录中。

算术（数学）。为了解决复杂的数学问题，大语言模型可以被增强以分解问题并逐步调用计算器工具。我们使用GSM8K-XL数据集（Hao et al., 2023年）评估这一用例，该数据集包含8.5K个高质量的小学数学问题。如预期，计算器的执行时间很短，平均为0.2毫秒。由于我们必须提示大语言模型逐步解决问题的演示，因此调用计算器时上下文长度相当大。

基于知识的问答 (QA)。为了允许

预训练的大语言模型访问更广泛的知识集，可以增强大语言模型，使其调用基于知识的问答工具，从丰富的数据集中检索信息。我们使用 Multihop QA 维基百科（Yang et al., 2018年）数据集来评估这一用例。尽管这些问答工具的问题上上下文长度更长，但由于与维基百科的非确定性网络通信延迟，它们提供相对快速但可变的执行时间。

虚拟环境 (VE)。大语言模型可以被增强以与虚拟环境交互，通过逐步执行完成复杂任务（Gu et al., 2022; Huang et al., 2022; Hu et al., 2023年）。为了评估 VE，我们使用ALFWorld数据集（Shridhar et al., 2021年），这是一个交互式环境，将文本描述和命令与物理实体机器人模拟对齐。由于是一个本地执行且具身化的基于文本的环境，VE交互具有短暂且稳定的拦截时间。由于需要提示大语言模型理解VE动作序列的指令，上下文长度较长。

聊天机器人。大语言模型的一个常见任务是聊天机器人，它通过一系列聊天与人类交互。当人类收到大语言模型的聊天回复时，大语言模型的生成序列实质上被拦截了。大语言模型的生成会在人类发送后续聊天消息时恢复。在此过程中，聊天历史必须作为整个聊天序列的上下文保留。我们使用ShareGPT数据集（Zheng et al., 2023年），其中包含众包的真实用户 ChatGPT对话来评估聊天机器人。我们将人类响应时间（即，拦截时间）估计为人类对前一条聊天回复的扫描时间（Bryshaert, 2019年）与下一条提示的打字时间（Ma等人, 2015）之和。由于人类提示的多样性，上下文长度变化很大。其拦截时间也存在高方差。

图像生成。大语言模型可以被增强以生成一张图像或一系列图像，当人类通过多个提示逐步细化特征时（例如，为面部描绘添加细节）。这涉及两次拦截：一次用于触发图像生成模型，另一次用于接收用户的后续提示。为了理解这个用例，我们使用ChatGPT创建了一个数据集，生成一系列图像生成提示，每个提示触发对Stable Diffusion模型（Rombach等人, 2021年）的调用。由于执行扩散模型的差异以及人类响应时间的差异，此工作负载的拦截时间具有高变异性。平均上下文长度比聊天机器人拦截更短，因为扩散模型的输入提示更小。

文本转语音 (TTS)。正如聊天- GPT Whisper API 支持所示 (Brockman et al., 2023年), 文本转语音可以与一个大语言模型集成，与人类进行通信。类似于我们的图像生成数据集，我们使用聊天-

GPT to generate a series of prompts, each triggering a call to the Bark TTS model (AI, 2023). Similar to image generation, we estimate the human response time but measure actual TTS execution time. TTS’s behavior also has high variation in interception time due to variations in model execution time and user response time.

Summary and insights. Overall, we find LLM interception time is highly dependent on augmentation types and exhibits a clear difference between short-running (Math, QA, VE) and long-running (Chatbot, Image, TTS) ones. The short-running augmentations are fully automated with no human interaction. Most also exhibit small variations (except for QA, which interacts with the network). Augmentations that involve human interaction and/or call another large model are long-running with high variations. The short/long interception times and their variation imply that no single LLM interception strategy is universally applicable, but an inference system could utilize augmentation type as a hint for interception time. Context lengths for all the augmentations are large, implying potentially high GPU memory wastage. Context lengths also see high variation, complicating the handling of them at runtime.

3 Existing LLM Inference Systems

This section discusses related work in LLM inference systems and why simple extensions to them are insufficient for interception handling, as shown in Figure 1.

3.1 LLM Inference Systems

Recent years have seen the rise of inference systems that are designed specifically for LLMs. Unlike INFERCEPT, none of the existing inference systems are designed for handling interceptions, and all suffer from unoptimized performance, as we will show in Section 3.2.

Some works aim to improve overall inference throughput with better scheduling strategies. For example, Orca (Yu et al., 2022) proposes iteration-level scheduling where, at the end of each model forward pass, Orca’s scheduler is invoked to form a new batch for the next forward pass. Such iteration-level scheduling strategies improve GPU utilization, resulting in higher inference throughput. INFERCEPT also adopts iteration-level scheduling but with a unique and new scheduling strategy — at each iteration, INFERCEPT makes a decision that minimizes GPU memory wastage for requests intercepted or resumed in that iteration based on their interception properties. Besides FCFS, which Orca and others use, alternative request scheduling policies such as a multi-level feedback queue in FastServe (Wu et al., 2023) have been proposed. We leave the comparison of these scheduling policies to future work.

Another optimization target is GPU memory usage.

vLLM (Kwon et al., 2023) proposes the concept of paged attention, where the KV cache is treated as virtual memory that can map to non-contiguous physical GPU memory. Because of the flexibility in virtual memory, vLLM achieves better utilization of GPU memory, thereby improving overall inference performance. INFERCEPT also utilizes paged attention to efficiently use GPU physical memory. Unlike vLLM, which is agnostic to interceptions, INFERCEPT adaptively preserves, discards, or swaps the KV cache of intercepted requests to minimize memory waste. Other memory-efficient techniques, such as prefix sharing in SGLang (Zheng et al., 2023b) and Preble (Srivatsa et al., 2024), are orthogonal and can be added to INFERCEPT.

Yet another set of LLM inference systems focuses on the imbalanced computation needs between the prefill and decoding stages. Sarathi (Agrawal et al., 2023) is a system that proposes the technique of chunked prefill, which splits prompt tokens into chunks, each merged with other decoding requests to form one iteration’s batch. INFERCEPT’s chunking splits sequences for recomputation and swapping to fully utilize GPU and GPU-CPU-link resources while minimizing memory waste. Other proposals on prefill-decoding optimization like DistServe (Zhong et al., 2024) are orthogonal and can potentially be added to INFERCEPT.

3.2 Limitations of Inference Systems and Extensions

We now qualitatively and quantitatively analyze the limitations of existing LLM inference systems and naive extensions to them to support interceptions. We quantify the impact of these approaches on serving speed with our proposed *GPU memory waste* calculations.

Discard-based approaches. Today’s LLM inference systems are not designed for augmented LLMs. To serve LLMs with augmentations, such a system would need to treat the interception as the end of the request and re-initiate a request when the interception finishes. As today’s inference systems use the FCFS scheduling policy, they schedule these re-initiated requests at the end of the waiting queue and starve resumed requests. An easy fix to this scheduling problem is using an intercepted request’s original arrival time when re-inserting it into the waiting queue (we call this scheme ImprovedDiscard).

Discard and ImprovedDiscard both require the recomputation of all context tokens and incur GPU memory waste from two sources. First, recomputation consumes memory that is not used to produce any new tokens. Assuming the context of request i when interception j occurs has C_i^j tokens, each token’s KV cache occupies M memory, and recomputation takes $T_{fwd}(C_i^j)$ time, where T_{fwd} is a mapping from the number of scheduled tokens in a batch to the execution time of that iteration.

GPT 生成一系列提示，每个触发对 Bark TTS 模型的调用 (AI, 2023年). 类似于图像生成，我们估计人类响应时间但测量实际文本转语音执行时间。文本转语音的行为在拦截时间上也存在高度变化，原因是模型执行时间和用户响应时间的变化。

总结与见解。总体来看，我们发现大语言模型拦截时间高度依赖于增强类型，并且在短运行（数学、问答、虚拟环境）和长运行（聊天机器人、图像、文本转语音）之间呈现出明显差异。短运行增强完全自动化，无需人类交互。大多数增强的变异性也较小（除了问答，它需要与网络交互）。涉及人类交互和/或调用另一个大型模型的增强属于长运行，且变异性高。短运行/长运行的拦截时间及其变异性表明，没有任何单一的大语言模型拦截策略是普遍适用的，但推理系统可以利用增强类型作为拦截时间的提示。所有增强的上下文长度都很大，这暗示了潜在的GPU内存浪费。上下文长度也存在高变异性，使得运行时处理更为复杂。

3 现有大语言模型推理系统

本节讨论大语言模型推理系统中的相关工作，以及为什么简单的扩展不足以处理拦截，如图1所示。

3.1 大语言模型推理系统

近年来，专门针对大语言模型设计的推理系统兴起。与INFERCEPT不同，现有的推理系统都没有设计用于处理拦截，并且都存在性能未优化的缺陷，我们将在第3.2节中展示这一点。

一些工作旨在通过更好的调度策略提升整体推理吞吐量。例如，Orca (Yuet al., 2022) 提出了迭代级调度，即在每次模型前向传播结束时，Orca的调度器被调用为下一次前向传播形成新的批次。这种迭代级调度策略提高了GPU利用率，从而获得更高的推理吞吐量。INFERCEPT也采用了迭代级调度，但采用了独特且新颖的调度策略——在每个迭代中，INFERCEPT会做出决策，以最小化该迭代中被拦截或恢复的请求的GPU内存浪费，这基于它们的拦截属性。除了Orca和其他系统使用的先来先服务策略外，还提出了替代的请求调度策略，例如FastServe (Wu et al., 2023年) 中的多级反馈队列。我们将这些调度策略的比较留作未来工作。

另一个优化目标是GPU内存使用。

vLLM (Kwon et al., 2023年) 提出了分页注意力的概念，其中KV缓存被视为虚拟内存，可以映射到非连续的物理GPU内存。由于虚拟内存的灵活性，vLLM实现了更好的GPU内存利用率，从而提高了整体推理性能。InferCept也利用分页注意力来高效使用GPU物理内存。与对拦截无感知的vLLM不同，InferCept自适应地保留、丢弃或交换被拦截请求的KV缓存，以最小化内存浪费。其他内存高效技术，如SGLang (Zheng et al., 2023b) 和Preble (Srivatsa et al., 2024) 中的前缀共享，是正交的，可以添加到InferCept中。

另一组大语言模型推理系统关注预填充和解码阶段之间不平衡的计算需求。Sarathi (Agrawal et al., 2023年) 是一个提出分块预填充技术的系统，它将提示令牌分割成多个块，每个块与其他解码请求合并形成一个迭代的批次。InferCept的分块将序列分割用于重计算和交换，以充分利用GPU和GPU-CPU链路资源，同时最小化内存浪费。其他关于预填充-解码优化的提议，如DistServe (Zhong et al., 2024)，是正交的，并且可以添加到InferCept中。

3.2 推理系统和扩展的局限性

我们现在定性和定量地分析现有大语言模型推理系统的局限性以及为支持拦截而对它们进行的简单扩展。我们通过提出的*GPU内存浪费*计算来量化这些方法对服务速度的影响。

基于丢弃的方法。如今的大语言模型推理系统并非为增强型LLM设计。为了为带有增强的LLM提供服务，这样的系统需要将拦截视为请求的结束，并在拦截完成时重新发起请求。由于如今的推理系统使用FCFS调度策略，它们将这些重新发起的请求调度到等待队列的末尾，从而使恢复的请求饥饿。对此调度问题的一个简单修复是在将拦截的请求重新插入等待队列时使用其原始到达时间（我们称此方案为ImprovedDiscard）。

Discard 和 ImprovedDiscard 都需要重计算所有上下文令牌，并且会因两个来源导致 GPU 内存浪费。首先，重计算会消耗内存，而这些内存并未用于生成任何新令牌。假设请求 i 在拦截 j 发生时其上下文包含 C_i^j 个令牌，每个令牌的 KV 缓存占用 M 内存，且重计算需要 $T_{fwd}(C_i^j)$ 时间，其中 T_{fwd} 是从批次中已调度令牌数量到该迭代执行时间的映射。

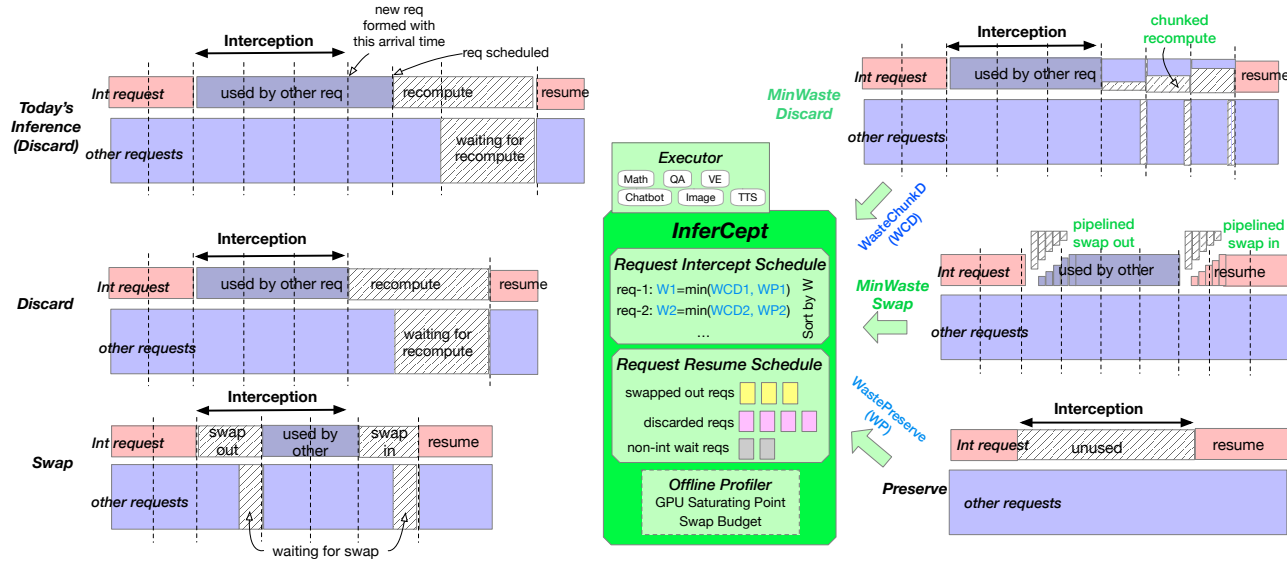


Figure 1: INFERCEPT and Alternative Approaches. Vertical: GPU resources occupied or wasted by intercepted requests and other normal ones. Horizontal: timeline divided by iterations (dotted vertical lines). Hatch parts represent memory waste. INFERCEPT adaptively combines preserve, our optimized discard, and our optimized swap. All green parts represent INFERCEPT's techniques.

The memory wasted for recomputation is $T_{fwd}(C_i^j) \times C_i^j \times M$. Second, recomputing the context increases iteration time, as an iteration now needs to finish all the recomputations alongside model forwarding of other running requests. The memory occupied by the other running requests is wasted during the additional iteration time, $T_{fwd}(C_i^j)$. Thus, the memory waste for this reason is $T_{fwd}(C_i^j) \times C_{other} \times M$, where C_{other} is the sum of the contexts of other requests. The total memory waste for Discard and ImprovedDiscard is:

$$\text{WasteDiscard}_i^j = T_{fwd}(C_i^j) \times C_i^j \times M + T_{fwd}(C_i^j) \times C_{other} \times M \quad (1)$$

In practice, our evaluation shows that Discard incurs a GPU resource wastage (GB*min) of 27%, and 37-40% of the total model forwarding time is spent on recomputation with a mixed workload containing all six augmentations in §2.

Preserve-based approach. Instead of discarding, one could preserve the context of a request when interception happens. This Preserve strategy avoids recomputation cost and can immediately resume a request when the interception finishes. However, the preserved context wastes GPU memory while the request remains paused. The preserve waste for request i when interception j occurs is the duration of that interception, T_{INT}^j , multiplied by the amount of GPU memory held by the request's context.

$$\text{WastePreserve}_i^j = T_{INT}^j \times C_i \times M \quad (2)$$

Our measured GPU memory waste with real interceptions is surprisingly high: nearly half of the GPU memory is occupied by interrupted requests more than 60% of the time.

Swap-based approach. To avoid recomputation overhead in Discard and memory waste in Preserve, one potential technique is to swap all context data from GPU to CPU memory when an interception happens and swap it back to GPU when the interception ends. The straightforward implementation of Swap performs swapping in a synchronous manner by launching CUDA kernels for moving data out/in and letting computation utilize the relevant memory space. Swapping can take an extremely long time with huge contexts because of the volume of data to be moved and the kernels to be launched. To elaborate on the latter, multiple kernels need to be launched, each for a discontinuous physical memory region. With PagedAttention (Kwon et al., 2023), the context of an intercepted request can scatter across many physical memory regions, causing high kernel launch overhead.

Like Discard, Swap incurs memory waste from two sources. First, the memory space being swapped is wasted during swapping, accounting for $T_{swap}(C_i^j) \times C_i^j \times M$, where T_{swap} is a mapping from the number of tokens to swap to the corresponding swapping latency. Second, Swap could increase the iteration time, causing all running requests to wait for swapping to finish and do nothing. This waste is $T_{swap}(C_i^j) \times C_{other} \times M$. The total waste doubles to account for swapping in and out:

$$\text{WasteSwap}_i^j = 2 \times T_{swap}(C_i^j) \times C_{batch} \times M \quad (3)$$

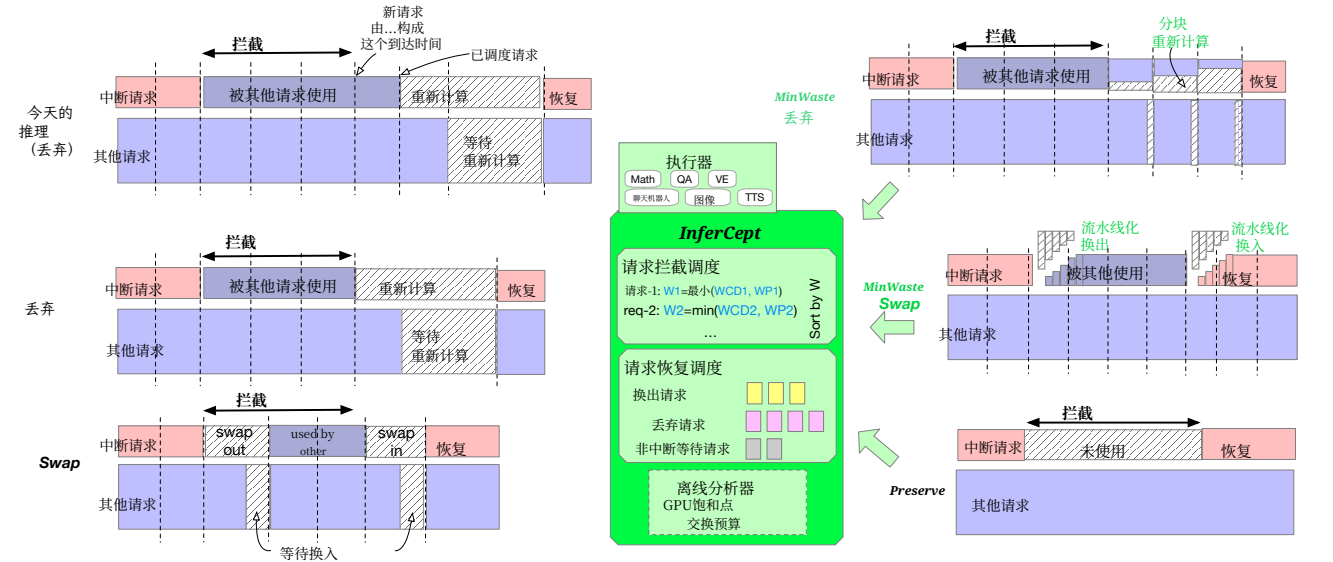


图1: INFERCEPT 与替代方法。纵向：被拦截的请求和其他正常请求占用或浪费的GPU资源。横向：按迭代（虚线竖线）划分的时间线。阴影部分表示内存浪费。INFERCEPT 自适应地结合了Preserve、我们优化的丢弃以及我们优化的交换。所有绿色部分表示INFERCEPT的技术。

重计算导致的内存浪费为 $T_{fwd}(C_i^j) \times C_i^j \times M$ 。其次，重计算上下文会增加迭代时间，因为一次迭代现在需要完成所有重计算，同时还要对其他正在运行的请求进行模型前向传播。在额外的迭代时间 $T_{fwd}(C_i^j)$ 内，其他正在运行请求所占用的内存被浪费。因此，这一原因导致的内存浪费为 $T_{fwd}(C_i^j) \times C_{other} \times M$ ，其中 C_{other} 是其他请求上下文的总和。Discard 和 ImprovedDiscard 的总内存浪费为：

$$\text{WasteDiscard}_i^j = T_{fwd}(C_i^j) \times C_i^j \times M + T_{fwd}(C_i^j) \times C_{other} \times M \quad (1)$$

在实际中，我们的评估表明，丢弃会导致27%的GPU资源浪费（GB*分钟），并且在包含 § 2 中所有六种增强的混合工作负载下，总模型前向传播时间的37%–40%花费在重计算上。

基于保留的方法。当拦截发生时，与其丢弃请求，不如保留其上下文。这种保留策略避免了重计算开销，并能在拦截结束后立即恢复请求。然而，当请求保持暂停状态时，保留的上下文会浪费GPU内存。对于请求 i ，当拦截 j 发生时，保留浪费的大小等于该拦截的持续时间 T_{INT}^j 乘以请求上下文所占用的GPU内存量。

$$\text{WastePreserve}_i^j = T_{INT}^j \times C_i \times M \quad (2)$$

我们实测的真实拦截场景下的GPU内存浪费高得惊人：超过60%的时间里，近一半的GPU内存被中断的请求所占用。

基于交换的方法。为了避免丢弃中的重计算开销以及保留中的内存浪费，一种潜在技术是在拦截发生时将所有上下文数据从GPU交换到CPU内存，并在拦截结束时将其交换回GPU。交换的直接实现通过启动CUDA内核来移入/移出数据，并让计算利用相应的内存空间，以同步方式执行交换。由于待移动的数据量和待启动的内核数量巨大，交换可能花费极长时间。具体来说，需要为每个不连续物理内存区域启动多个内核。使用PagedAttention (Kwon et al., 2023年)，被拦截请求的上下文可能分散在许多物理内存区域中，导致较高的内核启动开销。

与丢弃类似，Swap也会因两个原因导致内存浪费。首先，正在交换的内存空间在交换期间被浪费，占 $T_{swap}(C_i^j) \times C_i^j \times M$ ，其中 T_{swap} 是从要交换的令牌数到相应交换延迟的映射。其次，Swap可能会增加迭代时间，导致所有正在运行的请求等待交换完成而无所事事。这种浪费是 $T_{swap}(C_i^j) \times C_{other} \times M$ 。总浪费翻倍以计入换入换出的部分：

$$\text{浪费Swap}_i^j = 2 \times T_{swap}(C_i^j) \times C_{batch} \times M \quad (3)$$

Our evaluation shows that Swap wastes 26% GPU resources, and over 25% of the total workload time is spent on waiting for swapping for the mixed workload.

4 INFERENCEPT

This section presents INFERENCEPT, first with our improved swap and recomputation mechanisms, then with our intercepting and resuming request scheduling, and ending with interception duration estimation and implementation details. Figure 1 shows INFERENCEPT’s overall architecture.

4.1 Swap Pipelining and Chunking

We discuss how INFERENCEPT avoids swap memory waste.

Swap pipelining and overlapping. Swap performs swapping in a serial manner and blocks foreground computation for the entire swapping time. To mitigate this problem, we propose to perform swapping in a pipelined manner and in the background. Specifically, INFERENCEPT views each model layer’s swapping as one pipeline stage and pipelines kernel launching, data movement, and foreground model forwarding. For example, when we launch the swap kernel for layer $i + 2$, we move the context for layer $i + 1$, and layer i ’s context has been freed and used for normal forwarding.

Swap chunking. To further reduce memory waste, we propose to chunk swap-out and swap-in across multiple iterations so that in each iteration, the swap latency can be hidden by overlapping it with model forwarding. We obtain T_{fwd} through offline profiling and calculate T_{swap} based on the swapping bandwidth and the per-token memory requirements M . At iteration i with batch size B_i , we set $T_{swap}(N_i) = T_{fwd}(B_i)$ and calculate N_i . N_i , what we call the *swap limit*, indicates the number of tokens that can be swapped for free (*i.e.*, hidden behind model forwarding).

Determining swap-in and swap-out budget. At every iteration, there can be tokens waiting to be swapped out and swapped in. INFERENCEPT determines how much bandwidth to give to swap-out and to swap-in at each iteration by maximizing inference throughput (*i.e.*, the number of tokens that can be added to an iteration’s processing) while guaranteeing the following criteria: 1) the total amount of swap-in and swap-out tokens should be at most N_i ; 2) the amount of swapped-out memory should not exceed the amount of free CPU memory plus the swapped-in memory; and 3) the amount of swapped-in memory and memory for newly scheduled tokens (*i.e.*, our maximizing target) should not exceed the amount of swapped-out memory and free GPU memory. We use the obtained swap-in and swap-out budget when deciding the request interception schedule in §4.3.

4.2 Recomputation Chunking

Unlike Swap, which consumes GPU-CPU-link bandwidth and can be hidden from foreground GPU tasks, the recomputation of discarded tokens requires GPU resources that cannot be avoided. However, we observe that decoding and recomputation have complementary resource requirements, with decoding requiring less GPU core resource (for one query token) than recomputation (queries for the entire context length) for the same amount of memory. Thus, a batch of decoding requests usually cannot fill all GPU cores before running out of GPU memory. Mixing decoding and recomputation requests can increase GPU core utilization while staying within its memory boundary. The main challenge is how much recomputation to mix with decoding.

Discard and ImprovedDiscard impose the recomputation of an entire request in one iteration. Recomputation of a long context would add more computation burden than what a GPU can handle, causing significantly increased iteration time. As a result, long $T_{fwd}(C_i^j)$ causes huge *WasteDiscard* in Equation 1.

To mitigate this issue, we propose an adaptive technique that separates the recomputation of a context sequence into chunks, each added to one iteration without going beyond what the GPU can sustain. We observe that the number of query tokens that all GPU cores can process in parallel is limited and fixed for a given model architecture. We call this number the GPU *saturation point*, S , expressed as query token count. Processing more query tokens beyond S increases iteration time without improving serving throughput. INFERENCEPT obtains S from offline profiling and sets the chunk size as S minus the running group size.

The memory waste of our chunked recomputation mechanism can be calculated in two parts, similar to Equation 1. For the recomputation itself, we add one chunk of memory waste per iteration that lasts until all chunks have been recomputed, which essentially cuts the waste of Discard’s all-recomputation-at-once scheme by half, *i.e.*, $T_{fwd}(C_i^j) \times C_i^j \times M/2$, as illustrated in Figure 1. The second part of waste comes from other requests’ occupied memory during recomputation-added iteration time. This waste amounts to $n \times T_{fwd}(\frac{C_i^j}{n}) \times C_{other} \times M$, where n is the number of iterations to finish recomputing the request and $T_{fwd}(\frac{C_i^j}{n})$ is the per-chunk added iteration time. Adding these two parts, we have the total recomputation memory waste of chunked recomputation as

$$\text{WasteChunkD}_i^j = \frac{T_{fwd}(C_i^j) \times C_i^j \times M}{2} + n \times T_{fwd}(\frac{C_i^j}{n}) \times C_{other} \times M \quad (4)$$

Compared to Equation 1, the left term (recomputation it-

我们的评估显示, Swap浪费了26%的GPU资源, 并且超过25%的总工作负载时间用于等待交换混合工作负载。

4 INFERENCEPT

本节介绍INFERENCEPT, 首先是我们改进的交换与重计算机制, 然后是拦截和恢复请求调度, 最后是拦截持续时间估计和实现细节。图1展示了INFERENCEPT的整体架构。

4.1 交换流水线与分块

我们讨论INFERENCEPT如何避免交换内存浪费。

交换流水线与重叠。 Swap以串行方式执行交换, 并在整个交换时间内阻塞前台计算。为缓解此问题, 我们提出以流水线方式在后台执行交换。具体来说, INFERENCEPT将每个模型层的交换视为一个流水线阶段, 并将内核启动、数据移动和前台模型前向传播流水线化。例如, 当我们为层 $i + 2$ 启动交换内核时, 我们移动层 $i + 1$ 的上下文, 而层 i 的上下文已被释放并用于正常转发。

交换分块。 为了进一步减少内存浪费, 我们提出将交换出和交换入分散到多个迭代中执行, 从而在每次迭代中, 交换延迟可以通过与模型前向传播重叠来隐藏。我们通过离线剖析获得 T_{fwd} , 并根据交换带宽和每令牌内存需求 M 计算 T_{swap} 。在批次大小为 B_i 的迭代 i 中, 我们设置 $T_{swap}(N_i) = T_{fwd}(B_i)$ 并计算 N_i 。 N_i , 即所谓的交换上限, 表示可以免费交换的令牌数 (即, 隐藏在模型前向传播之后)。

确定交换入和交换出预算。 在每次迭代中, 可能存在等待交换出和交换入的标记。INFERENCEPT 通过最大化推理吞吐量 (即, 每次迭代处理中可添加的令牌数) 来确定每次迭代分配给交换出和交换入的带宽, 同时保证以下条件: 1) 交换入和交换出的标记总量应最多为 N_i ; 2) 交换出的内存量不应超过空闲CPU内存加上交换入的内存量; 以及3) 交换入的内存量与新调度标记的内存 (即, 我们要最大化的目标) 不应超过交换出的内存量与空闲GPU内存之和。我们使用所得的交换入和交换出预算, 在 §4.3 中决定请求拦截调度。

4.2 重计算分块

与Swap不同, 后者会消耗GPU-CPU链路带宽并可隐藏于前台GPU任务后, 而对丢弃的令牌进行重计算则需要占用无法避免的GPU资源。然而, 我们发现解码与重计算在资源需求上具有互补性: 对于相同大小的内存, 解码 (针对一个查询令牌) 所需的GPU内核资源少于重计算 (针对整个上下文长度的查询)。因此, 一批解码请求通常在耗尽GPU内存之前无法填满所有GPU核心。混合解码请求与重计算请求可以在内存边界内提高GPU核心利用率。主要挑战在于解码中应混合多少重计算。

丢弃和ImprovedDiscard 要求在一个迭代内重计算整个请求。对长上下文进行重计算会带来超出 GPU 处理能力的计算负担, 导致迭代时间显著增加。因此, 长 $T_{fwd}(C_i^j)$ 会导致公式1中的大幅增加。

为了缓解这个问题, 我们提出了一种自适应技术, 将上下文序列的重计算分割成多个块, 每个块添加到一次迭代中, 且不超过GPU的承载能力。我们观察到, 所有GPU核心能够并行处理的查询令牌数量是有限的, 并且在给定模型架构下是固定的。我们将这个数字称为GPU饱和点 S , 以查询令牌数量表示。处理超过 S 的查询令牌会增加迭代时间, 但不会提高服务吞吐量。InferCept通过离线剖析获取 S , 并将块大小设置为 S 减去运行组大小。

我们的分块重计算机制的内存浪费可以分为两部分, 类似于方程1。对于重计算本身, 每次迭代我们增加一块内存浪费, 该浪费持续到所有块都被重计算完毕, 这实质上将丢弃的一次性全部重计算方案的浪费减半, 即, $T_{fwd}(C_i^j) \times C_i^j \times M/2$, 如图1所示。第二部分浪费来自重计算增加的迭代时间内其他请求占用的内存。该浪费为 $n \times T_{fwd}(\frac{C_i^j}{n}) \times C_{other} \times M$, 其中 n 是完成重计算该请求所需的迭代次数, 而 $T_{fwd}(\frac{C_i^j}{n})$ 是每块增加的迭代时间。将这两部分相加, 我们得到分块重计算的总重计算内存浪费为

$$\text{WasteChunkD}_i^j = \frac{T_{fwd}(C_i^j) \times C_i^j \times M}{2} + n \times T_{fwd}(\frac{C_i^j}{n}) \times C_{other} \times M \quad (4)$$

与公式1, 左边项 (重计算它-

self) cuts the corresponding term of `Discard` by half, and the right term (other requests) is no larger than the right term of `Discard` because $n \times T_{fwd}(\frac{C_i^j}{n}) \leq T_{fwd}(C_i^j)$. Meanwhile, INFERCEPT increases GPU core utilization and further improves overall serving throughput.

4.3 Inter-Request Action Decision

Scheduling intercepted requests. We now discuss how INFERCEPT schedules intercepted requests using a hybrid of preserve, our improved swap, and our improved discard when considering multiple requests together. First, for each request i encountering interception j , we calculate its memory waste as the minimum of `WastePreserve` (Equation 2) and `WasteChunkDiscard` (Equation 4):

$$\text{Waste}_i^j = \min(\text{WastePreserve}_i^j, \text{WasteChunkD}_i^j) \quad (5)$$

Next, we sort all intercepted requests in descending order based on their memory waste. We swap out context from these requests according to this order until we run out of the swap-out budget. For the remaining paused requests, we preserve or discard their remaining context based on its decision from Equation 5.

Scheduling resumed and other waiting requests. At the beginning of each iteration, INFERCEPT determines what requests to insert into the batch for the iteration. To facilitate this decision, INFERCEPT maintains three queues: a running queue containing currently running requests in the system, a swap queue containing requests that have been resumed but are previously swapped out during the interception, and a waiting queue. The waiting queue contains discarded requests that have been resumed, new requests received by INFERCEPT that have never been served, and previously running requests that have been evicted due to the lack of GPU resources. The latter two types are the same as today’s inference systems. Each queue is sorted by its requests’ original arrival times.

At each iteration, we choose requests from the waiting queue in the FCFS order until the GPU saturation point is reached; FCFS ensures fairness and avoids starvation. Requests chosen can be any of the three aforementioned waiting types. If it is a discarded request, the iteration will perform the recomputation of the scheduled number of tokens. If this request has only been partially recomputed, it remains in the wait queue with the remaining tokens. At each iteration, we also choose requests from the swap queue in the FCFS order until the swap-in budget is reached. We maintain and schedule a separate swap queue because the swap-in budget is additional to GPU resources and should always be utilized by resumed requests as much as the budget allows.

4.4 Interception Duration Estimation

The calculation of memory waste for preserving context (Equation 2) requires interception duration. For interceptions with highly variable durations or those not profiled offline, we propose a dynamic estimation method by setting $\hat{T}_{INT} = t_{now} - t_{call}$ where t_{now} is the current time updated each iteration and t_{call} is when the last interception was initiated. Effectively, the longer a request is intercepted, the larger the estimated interception time. Our evaluation shows that INFERCEPT using this estimation method achieves 93% of the performance compared with using an oracle providing exact interception durations in mixed workload.

4.5 Implementation

INFERCEPT comprises four key components: a scheduling policy, waste calculation, chunked recomputation and swapping, and diverse augmentation support. We implement INFERCEPT on top of vLLM (Kwon et al., 2023) to leverage its PagedAttention technique for regular LLM memory management. Most of our techniques are highly modular and orthogonal to optimizations designed for non-intercepted LLMs. Thus, INFERCEPT can be potentially integrated into other LLM serving systems like DeepSpeed (Aminabadi et al., 2022), Orca (Yu et al., 2022), and TensorRT-LLM (Vaidya et al., 2023).

5 Evaluation Results

We compare INFERCEPT to vLLM (with no modifications), `ImprovedDiscard`, `Preserve`, and `Swap`, as discussed in §3.2. To evaluate INFERCEPT and the baselines, we augment the 6B-parameter GPT-J model (Wang & Komatsuzaki, 2021), the 13B Vicuna model (Zheng et al., 2023a), and the 70B Llama3 model (Meta, 2024) with workloads presented in §2. We run augmented GPT-J on one NVIDIA A100 GPU. For Vicuna, we use two environments: running on a single A100 GPU and distributed on two A100 GPUs with tensor parallelism. For Llama3, we distribute it on four A100 GPUs with tensor parallelism. To mimic real-world serving scenarios that often receive different types of requests, we use a request dataset that merges the six augmentations presented in §2 by uniformly sampling requests from them.

5.1 End-to-End Performance

We first compare the end-to-end performance of INFERCEPT and the baselines. Following recent LLM inference research papers (Kwon et al., 2023; Yu et al., 2022), we first report the serving throughput as normalized latency (*i.e.*, the median of every request’s end-to-end latency divided by its output length) when varying request load (number of

自身)将丢弃的对应项减半，并且右边项（其他请求）不大于丢弃的右边项，因为 $n \times T_{fwd}(\frac{C_i^j}{n}) \leq T_{fwd}(C_i^j)$ 。与此同时，InferCept提高了GPU核心利用率，并进一步提升了整体服务吞吐量。

4.3 请求间动作决策

调度被拦截的请求。 我们现在讨论如何 INFERCEPT 使用 `Preserve`、我们改进的交换和改进的丢弃的混合策略来调度被拦截的请求，同时考虑多个请求。首先，对于每个请求 i 遇到拦截 j ，我们计算其内存浪费为 `WastePreserve`（公式 2）和 `WasteChunkDiscard`（公式4）：

$$\text{Waste}_i^j = \min(\text{WastePreserve}_i^j, \text{WasteChunkD}_i^j) \quad (5)$$

接下来，我们根据所有被拦截的请求的内存浪费按降序排序。我们按照此顺序从这些请求中换出上下文，直到用完换出预算。对于剩余的暂停请求，我们根据公式中的决策 `Preserve` 或丢弃其剩余上下文5。

调度已恢复的请求和其他等待的请求。 在每次迭代开始时，INFERCEPT 决定将哪些请求插入到该迭代的批次中。为了便于决策，INFERCEPT 维护三个队列：一个包含系统中当前正在运行的请求的运行队列，一个包含已恢复但在拦截期间之前被换出的请求的交换队列，以及一个等待队列。等待队列包含已恢复的已丢弃请求、INFERCEPT 收到但从未服务过的新请求，以及由于缺乏 GPU 资源而被逐出的先前正在运行的请求。后两种类型与当今的推理系统相同。每个队列按其请求的原始到达时间排序。

在每个迭代中，我们按照先来先服务顺序从等待队列中选择请求，直到达到GPU饱和点；先来先服务确保了公平性并避免了饥饿。选中的请求可以是前述三种等待类型中的任意一种。如果是被丢弃的请求，则该迭代将执行计划令牌数的重计算。如果该请求仅部分重计算完成，它将带着剩余令牌继续留在等待队列中。在每个迭代中，我们还按照先来先服务顺序从交换队列中选择请求，直到达到换入预算。我们维护并调度一个独立的交换队列，因为换入预算是对GPU资源的补充，应始终由恢复的请求尽可能多地利用该预算。

4.4 拦截持续时间估计

计算保留上下文的内存浪费（公式2）需要拦截持续时间。对于持续时间高度可变或未在离线状态下分析的拦截，我们提出了一种动态估计方法，通过设定 $T_{INT} = t_{now} - t_{call}$ ，其中 t_{now} 是每个迭代更新的当前时间， t_{call} 是上次发起拦截的时间。实际上，请求被拦截的时间越长，估计的拦截时间就越大。我们的评估表明，使用这种估计方法的InferCept在混合工作负载下，其性能达到了使用提供精确拦截持续时间的预言机时的93%。

4.5 实现

INFERCEPT 包含四个关键组件：调度策略、浪费计算、分块重计算与交换，以及多样化增强支持。我们实现 INFERCEPT 基于vLLM (Kwon et al.,2023)，利用其 PagedAttention技术进行常规大语言模型内存管理。我们的大多数技术具有高度模块化，并且与为非拦截大语言模型设计的优化正交。因此，INFERCEPT 可以潜在地集成到其他大语言模型服务系统中，如DeepSpeed (Aminabadi et al., 2022)，Orca (Yuet al., 2022)，以及 TensorRT-LLM (Vaidya et al., 2023)。

5 评估结果

我们比较INFERCEPT 与vLLM（未经修改），`ImprovedDiscard`，`Preserve`，以及`Swap`，如 § 3.2所述。为了评估INFERCEPT和基线，我们增强6B参数GPT-J模型 (Wang & Komatsuzaki, 2021年)，13B Vicuna模型 (Zhenget al.,2023a)，以及70B Llama3模型 (Meta, 2024) 使用 § 2中介绍的工作负载。我们使用增强后的GPT-J运行在单个NVIDIA A100 GPU上。对于Vicuna，我们使用两种环境：在单个A100 GPU上运行，以及在两个A100 GPU上分布运行，采用张量并行。对于Llama3，我们在四个A100 GPU上分布运行，采用张量并行。为了模拟真实世界中经常接收不同类型请求的服务场景，我们使用一个请求数据集，通过从 § 2 中均匀采样请求，合并了六种增强。

5.1 端到端性能

我们首先比较InferCept-CEPT 和基线系统的端到端性能。遵循近期的大语言模型推理研究论文 (Kwon et al., 2023; Yu et al., 2022)，我们首先报告在改变请求负载时，以归一化延迟表示的服务吞吐量（即，每个请求的端到端延迟中位数除以输出长度），该负载为（数量

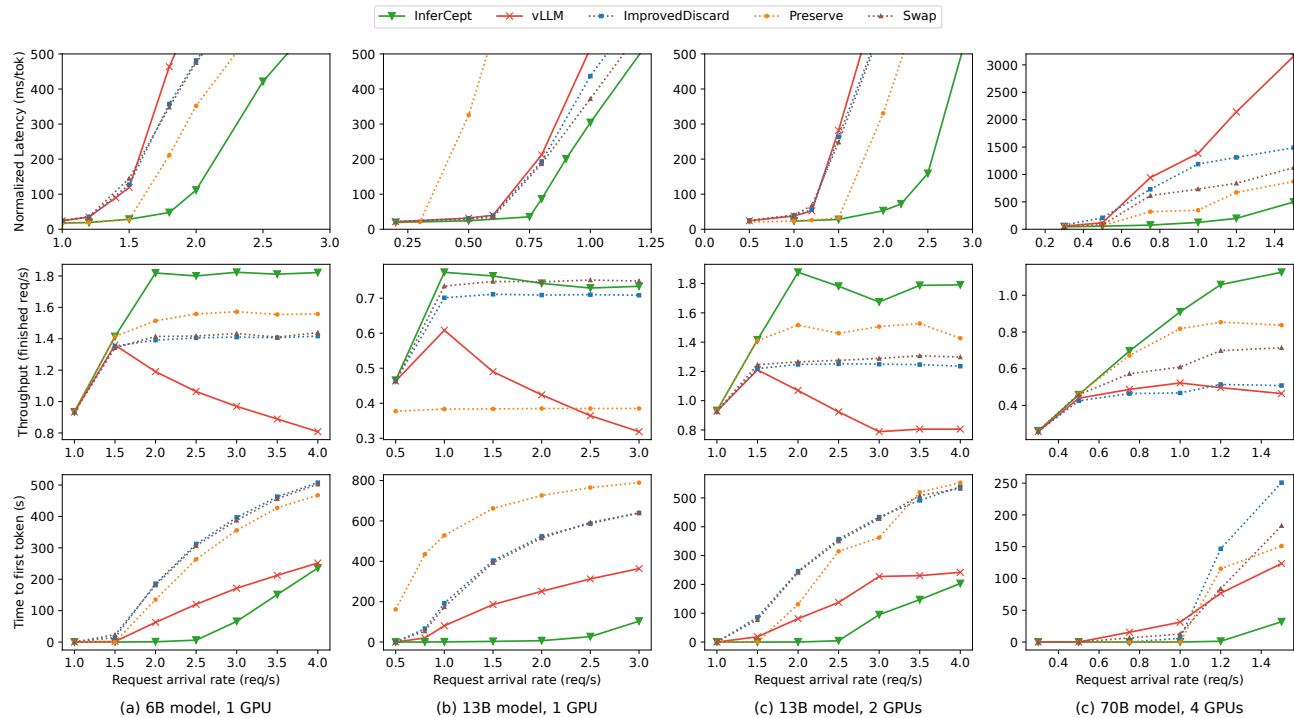


Figure 2: **End-to-end Performance on Mixed Workload.** *First Row: Normalized Latency.* Lower right is better, i.e., sustains higher serving load. *Second Row: Throughput.* Expressed as completed requests per second. Higher is better. *Third Row: Time-to-first-token (TTFT).* Lower is better, i.e., shorter response time.

requests arrived per second). We also remove a request’s intercepted time from its end-to-end latency because it is the same across all serving systems. Under the same normalized latency, a higher-throughput system should sustain a higher request rate. Apart from normalized latency, we also report the number of finished requests per second (i.e., throughput) and the time from request arrival to the first generated token (i.e., TTFT). These metrics are important in meeting service-level objectives and system response time (Zhong et al., 2024; Hu et al., 2024; Patel et al., 2024). Figure 2 shows these results for INFERCEPT and the baselines on all three models. For Vicuna-13B, we report both a single GPU and two GPU executions.

6B model results. INFERCEPT outperforms baselines across all metrics. INFERCEPT sustains up to $1.6\times$ higher request arrival rate at the same normalized latency compared to vLLM. Under the same request rate, INFERCEPT’s normalized latency is $1.9\times$ - $5.7\times$ lower than vLLM. INFERCEPT serves $1.7\times$ higher request arrival rate than vLLM with the smallest TTFT. INFERCEPT outperforms all of the baselines mainly because of our min-waste-based schedule. Moreover, INFERCEPT improves the recomputation and swap mechanisms over ImprovedDiscard and Swap with chunking and pipelining. Compared to them, INFERCEPT eliminates over 60% of GPU waste caused by recomputation and 96% of GPU waste for swapping.

The vanilla vLLM suffers from high recomputation waste and delays in scheduling resumed requests per FCFS policy, resulting in worse throughput and normalized request latency. Its TTFT is better than other baselines and close to INFERCEPT because it puts intercepted requests to the end of the wait queue, which is a round-robin schedule. This results in the system primarily handling requests before their first interception, essentially allowing TTFT not to be affected by interceptions. ImprovedDiscard maintains an intercepted request’s original arrival time. Thus, both its normalized latency and throughput are better than vanilla vLLM. However, ImprovedDiscard still incurs the same recomputation overhead and underperforms INFERCEPT on all metrics. Preserve is the best among all the baselines regarding normalized latency and throughput, showing that the preserving overhead for our mixed workload with small models is lower than the recomputation overhead for most requests. Yet, Preserve is worse than INFERCEPT on all metrics because of INFERCEPT’s dynamic min-waste scheduling and improved swapping and recomputation. The overall results for Swap are similar to ImprovedDiscard. Both baselines avoid GPU memory waste during interceptions. While ImprovedDiscard causes running requests to wait for the recomputation requests in an iteration, Swap causes running requests to wait for swapping out/in requests in an iteration. These overheads have similar effects on the final performance metrics.

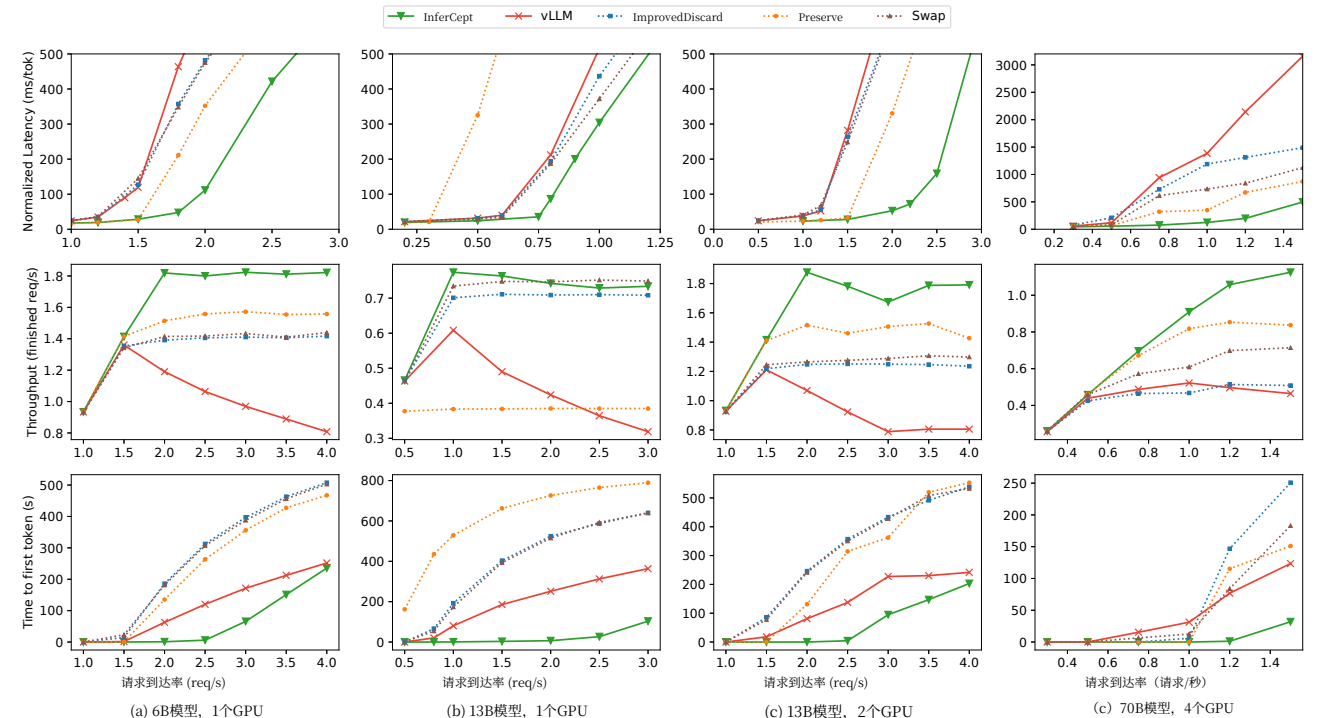


图 2：混合工作负载下的端到端性能。第一行：归一化延迟。越往右下角越好，即能承受更高的服务负载。第二行：吞吐量。以每秒完成请求数表示。越高越好。第三行：首令牌时间 (TTFT)。越低越好，即响应时间更短。

每秒到达的请求数)。我们还从每个请求的端到端延迟中移除了被拦截时间，因为所有服务系统的该时间相同。在相同的归一化延迟下，吞吐量更高的系统应能维持更高的请求速率。除归一化延迟外，我们还报告每秒完成的请求数（即，吞吐量）以及从请求到达生成第一个令牌的时间（即，TTFT）。这些指标对于满足服务水平目标和系统响应时间至关重要（Zhong et al., 2024; Hu et al., 2024; Patelet et al., 2024）。图2展示了InferCept和基线系统在所有三个模型上的这些结果。对于Vicuna-13B，我们报告了单GPU和双GPU两种执行情况。

6B模型结果。 InferCept 在所有指标上均优于基线方法。在相同的归一化延迟下，InferCept 能承受的请求到达率比 vLLM 高出 $1.6\times$ 。在相同请求率下，InferCept 的归一化延迟比 vLLM 低 $1.9\times$ - $5.7\times$ 。InferCept 在最小 TTFT 下服务的请求到达率比 vLLM 高 $1.7\times$ 。InferCept 优于所有基线方法，主要是因为我们的最小浪费调度。此外，InferCept 通过分块和流水线改进了 ImprovedDiscard 和 Swap 中的重计算和交换机制。与它们相比，InferCept 消除了重计算导致的超过 60% 的 GPU 浪费，以及交换导致的 96% 的 GPU 浪费。

原始的 vLLM 存在高重计算浪费，并且按照先来先服务策略调度恢复的请求时会产生延迟，导致更差的吞吐量和归一化请求延迟。其 TTFT 优于其他基线，接近 InferCept，因为它将被拦截的请求放到等待队列的末尾，这属于轮询调度。这导致系统主要在请求首次拦截之前处理它们，从而使得 TTFT 不受拦截影响。ImprovedDiscard 保留了被拦截请求的原始到达时间，因此其归一化延迟和吞吐量均优于原始 vLLM。然而，ImprovedDiscard 仍然产生相同的重计算开销，并且在所有指标上均不如 InferCept。Preserve 在归一化延迟和吞吐量方面是所有基线中最好的，这表明对于包含小模型的混合工作负载，保留操作的开销低于大多数请求的重计算开销。然而，Preserve 在所有指标上均不如 InferCept，这是因为 InferCept 的动态最小浪费调度以及改进的交换和重计算。Swap 的整体结果与 ImprovedDiscard 类似。这两个基线都避免了拦截期间的 GPU 内存浪费。ImprovedDiscard 导致正在运行的请求在迭代中等待重计算请求，而 Swap 导致正在运行的请求在迭代中等待换出/换入请求。这些开销对最终性能指标的影响相似。

13B model results. When running the 13B model on a single GPU, INFERCEPT outperforms all the baselines in normalized latency but with smaller improvements than the 6B model. A larger model’s weights occupy more GPU memory, reducing space for the KV cache. Moreover, the increased normal forwarding time contributes to most of the normalized latency, limiting the room for improvement. Still, we serve up to $1.25\times$ higher request rates without a noticeable increase in the normalized latency and also sustain $3.1\times$ higher load with the smallest TTFT.

A more practical scenario for large model inference is a distributed execution environment. We evaluate the 13B model with tensor-parallelism (Shoeybi et al., 2019) on two GPUs. INFERCEPT outperforms all baselines by a larger margin than when running on a single GPU. INFERCEPT serves up to $1.8\times$ higher request arrival rates and a $1.6\times$ - $10\times$ improvement in normalized latency compared to vLLM. We obtain more benefits in the distributed setting because each GPU uses less memory for storing model weights and allocates more space for the KV cache. This allows more requests to be served concurrently and thus introduces more interceptions. As INFERCEPT minimizes GPU memory waste from these interceptions, the improvement over baselines becomes more pronounced in this setting.

70B model results. For the Llama3-70B model, INFERCEPT has substantial improvement over all baselines. INFERCEPT can sustain $2\times$ higher request arrival rate and achieve $1.3\times$ - $12\times$ lower normalized latency at the same RPS. We also achieve $2.4\times$ higher load than vLLM with the same TTFT. As model size increases, both vanilla vLLM and ImprovedDiscard experience significant recomputation costs. In contrast, Preserve and Swap perform better. This is attributed to the adoption of grouped query attention (GQA) (Ainslie et al., 2023), which compresses the attention KVs. This mechanism reduces inactive memory for Preserve and moderates data movement for Swap. By making optimal preemption decisions tailored to specific requests, INFERCEPT provides more benefits since the GPU waste is lowered significantly.

Single-augment workloads. Apart from the mixed workload, we evaluate INFERCEPT on two single-API workloads: QA and Chatbot (§2). INFERCEPT outperforms vLLM by up to $2.3\times$ and $1.9\times$ on normalized latency with QA and Chatbot respectively. INFERCEPT’s improvement with QA is larger because most QA API calls are short and have smaller waste when using preserve than discard.

5.2 Performance Deep Dive

To further understand INFERCEPT’s benefits, we break down its techniques by adding them one at a time to vanilla vLLM. Using the mixed workload, we report the normal-

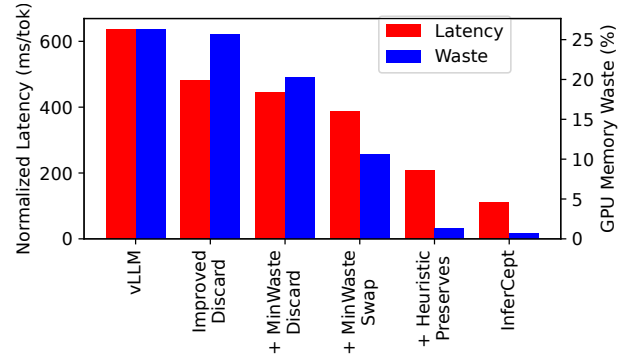


Figure 3: **INFERCEPT Technique Breakdown.** Each bar group adds one technique over its left bar group, with the leftmost being vanilla vLLM and the rightmost being the full INFERCEPT.

ized latency and GPU memory waste under the load of 2 requests per second for the 6B model in Figure 3. Other request loads have similar trends.

We first improve Discard (vanilla vLLM) by maintaining the request’s original arrival time as in ImprovedDiscard, which reduces normalized latency by 24.5%. We then add INFERCEPT’s recomputation chunking, resulting in 7.8% more improvement. Next, we add INFERCEPT’s budgeted swapping (discarding once the limit is reached). This results in an added improvement of 12.7%. Then, we add preserve and use a simple heuristic to decide between discard and preserve: discard for interactive (or long-running) interceptions and preserve for automated (short-running) interceptions. This addition has a 46.1% improvement, mainly because of the added support for Preserve. Finally, adding min-waste-based adaptive schedule (*i.e.*, the whole INFERCEPT) results in an additional 46.4% improvement. Each of INFERCEPT’s techniques reduces GPU memory waste, with the whole INFERCEPT only having 0.69% waste. This demonstrates the effectiveness of our min-waste preemption principle.

Note that INFERCEPT’s techniques’ benefits differ for different workloads. For example, our chunking and pipelining techniques for recomputation and swapping contribute 54% of total speedup for the single Chatbot workload. This is because Chatbot has a long interception duration, allowing for more requests to execute and, in turn, triggering more swap and recomputation.

6 Conclusion

We introduced INFERCEPT, an LLM inference framework optimized for interceptions during inference. By minimizing GPU memory waste as a unified objective, INFERCEPT delivers $1.6\times$ - $2\times$ higher request arrival rates and achieves $1.3\times$ - $12\times$ lower normalized latency compared to state-of-the-art inference systems.

13B模型结果。当在单个 GPU 上运行 13B 模型时，InferCept 在归一化延迟方面优于所有基线，但改进幅度小于 6B 模型。更大模型的权重占用更多 GPU 内存，从而减少了 KV 缓存的空间。此外，增加的正常转发时间占据了归一化延迟的大部分，限制了改进空间。尽管如此，我们可以在归一化延迟无明显增加的情况下服务高达 $1.25\times$ 倍的请求率，并且还能以最小的 TTFT 承载 $3.1\times$ 倍的负载。

大语言模型推理的一个更实际场景是分布式执行环境。我们使用张量并行（Shoeybi等人，2019年）在两块 GPU上评估13B模型。INFERCEPT比在单块GPU上运行时相比基线提升更大。与vLLM相比，INFERCEPT可提供高达 $1.8\times$ 倍的更高请求到达率，并实现 $1.6\times$ - $10\times$ 倍的归一化延迟改进。我们在分布式环境中获得了更多优势，因为每个GPU用于存储模型权重的内存更少，从而为KV缓存分配了更多空间。这使得更多请求能够并发处理，从而引入更多拦截。由于INFERCEPT将这些拦截导致的GPU内存浪费降至最低，因此在此设置中，相较于基线的改进更加显著。

70B模型结果。对于Llama3-70B模型，INFERCEPT在所有基线上均有显著改进。INFERCEPT能够承受 $2\times$ 倍更高的请求到达率，并在相同RPS下实现 $1.3\times$ - $12\times$ 倍的更低归一化延迟。在与vLLM具有相同TTFT的情况下，我们还能实现 $2.4\times$ 倍更高的负载。随着模型规模增大，原始vLLM和ImprovedDiscard都经历了显著的重新计算成本。相比之下，Preserve和Swap表现更好。这归功于采用了分组查询注意力（GQA）（Ainslie等人，2023年），该机制压缩了注意力KVs。该机制减少了Preserve的非活跃内存，并缓和了Swap的数据移动。通过针对特定请求做出最优抢占决策，INFERCEPT提供了更多收益，因为GPU浪费显著降低。

单一增强工作负载。除了混合工作负载，我们评估InferCept在两种单一API工作负载上：问答和聊天机器人（§2）。InferCept在归一化延迟上比vLLM最多提升 $2.3\times$ 倍和 $1.9\times$ 倍，与

问答和聊天机器人分别。InferCept对问答的提升更大，因为大多数问答API调用较短，且使用Preserve比丢弃时的浪费更小。

5.2 性能深度剖析

为了进一步了解INFERCEPT的优势，我们通过将其技术逐一添加到原生vLLM中进行分解。在混合工作负载下，我们报告了正常

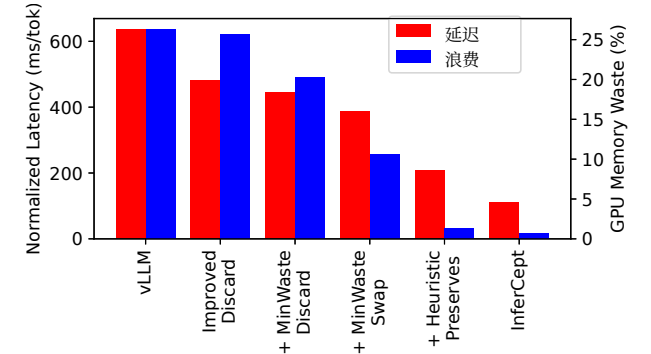


图3：InferCept 技术分解。每个柱状组在其左边的柱状组基础上增加一种技术，最左边是原生vLLM，最右边是完整的InferCept。

化延迟和GPU内存浪费，条件为6B模型在每秒2个请求的负载下，如图3所示。其他请求负载也有类似趋势。

我们首先通过保留请求的原始到达时间（如ImprovedDiscard）来改进Discard（原始vLLM），这使归一化延迟降低了24.5%。随后加入INFERCEPT的重计算分块，进一步提升了7.8%。接着，我们添加INFERCEPT的预算交换（达到限制后丢弃），这又带来了12.7%的改进。然后，我们加入Preserve，并使用简单的启发式规则来决定是丢弃还是保留：对交互式（或长运行）拦截进行丢弃，对自动化（短运行）拦截进行保留。这一改进提升了46.1%，主要归功于新增的Preserve支持。最后，加入基于最小浪费的自适应调度（即，完整的INFERCEPT），又额外实现了46.4%的提升。INFERCEPT的每一项技术都减少了GPU内存浪费，完整的INFERCEPT仅产生0.69%的浪费。这验证了我们最小浪费抢占原则的有效性。

需要注意的是，InferCept 的各项技术带来的效益因不同负载而异。例如，我们针对重计算和交换所采用的分块与流水线技术，在单个聊天机器人工作负载中贡献了总加速的54%。这是因为聊天机器人具有较长的拦截持续时间，允许更多请求执行，进而触发更多的交换和重计算。

6 结论

我们提出了 INFERCEPT，一种针对推理过程中拦截进行优化的大语言模型推理框架。通过将最小化 GPU内存浪费作为统一目标，与最先进的推理系统相比，INFERCEPT 实现了 $1.6\times$ - $2\times$ 倍的更高请求到达率，并将归一化延迟降低了 $1.3\times$ - $12\times$ 倍。

Acknowledgements

We would like to thank the anonymous reviewers for their tremendous feedback and comments, which have substantially improved the content and presentation of this paper. We are also thankful to Longfei Yun, Shravan Konduru, Hao-Ting Tso, Chujia Guo, Yu-shu Chen, Chace Zhu, Shivam Pansuria, and Katherine Guo for their contribution to the initial phase of this work. This material is based upon work supported by gifts from AWS, Google, and Meta. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of these institutions.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

Agrawal, A., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B. S., and Ramjee, R. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369*, August 2023.

AI, S. Bark: Text-to-speech model. <https://github.com/suno-ai/bark>, 2023.

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebron, F., and Sanghai, S. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, Singapore, December 2023.

Aminabadi, R. Y., Rajbhandari, S., Awan, A. A., Li, C., Li, D., Zheng, E., Ruwase, O., Smith, S., Zhang, M., Rasley, J., et al. Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, Dallas, Texas, November 2022. IEEE.

Baeza-Yates, R., Ribeiro-Neto, B., et al. *Modern Information Retrieval*, volume 463. ACM Press, New York, 1999.

Betker, J., Goh, G., Jing, L., Brooks, T., Wang, J., Li, L., Ouyang, L., Zhuang, J., Lee, J., Guo, Y., Man-

assra, W., Dhariwal, P., Chu, C., Jiao, Y., and Ramesh, A. Improving image generation with better captions. <https://cdn.openai.com/papers/dall-e-3.pdf>, 2024.

Brockman, G., Eleti, A., Georges, E., Jang, J., Kilpatrick, L., Lim, R., Miller, L., and Pokrass, M. Introducing chatgpt and whisper apis. <https://openai.com/blog/introducing-chatgpt-and-whisper-apis>, March 1 2023.

Brysbaert, M. How many words do we read per minute? a review and meta-analysis of reading rate. *Journal of memory and language*, 109:104047, 2019.

Chase, H. LangChain. <https://github.com/langchain-ai/langchain>, October 2022.

Chen, L., Chen, Z., Tan, B., Long, S., Gasic, M., and Yu, K. Agentgraph: Towards universal dialogue management with structured deep reinforcement learning. *arXiv preprint arXiv:1905.11259*, May 2019.

Costa-jussà, M. R., Cross, J., Çelebi, O., Elbayad, M., Heafield, K., Heffernan, K., Kalbassi, E., Lam, J., Licht, D., Maillard, J., Sun, A., Wang, S., Wenzek, G., Youngblood, A., Akula, B., Barrault, L., Gonzalez, G. M., Hansanti, P., Hoffman, J., Jarrett, S., Sadagopan, K. R., Rowe, D., Spruit, S., Tran, C., Andrews, P., Ayan, N. F., Bhosale, S., Edunov, S., Fan, A., Gao, C., Goswami, V., Guzmán, F., Koehn, P., Mourachko, A., Ropers, C., Saleem, S., Schwenk, H., and Wang, J. No language left behind: Scaling human-centered machine translation. *arXiv preprint arXiv:2207.04672*, July 2022.

Greyling, C. When using the chatgpt api, users will have to manage the context. <https://cobusgreyling.medium.com/when-using-the-chatgpt-api-users-will-have-to-manage-the-context-ba5869238913>, March 6 2023.

Gu, J., Stefani, E., Wu, Q., Thomason, J., and Wang, X. Vision-and-language navigation: A survey of tasks, methods, and future directions. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Dublin, Ireland, 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-long.524.

Hao, S., Liu, T., Wang, Z., and Hu, Z. Toolkengpt: Augmenting frozen language models with massive tools via tool embeddings. In *Advances in Neural Information Processing Systems 36*, New Orleans, Louisiana, December 2023.

Hu, C., Huang, H., Xu, L., Chen, X., Xu, J., Chen, S., Feng, H., Wang, C., Wang, S., Bao, Y., Sun, N., and Shan, Y. Inference without interference: Disaggregate llm inference for mixed downstream workloads. *arXiv preprint arXiv:2401.11181*, January 2024.

致谢

我们感谢匿名审稿人提供的宝贵反馈和意见，这些反馈极大地改进了本文的内容和呈现。我们也感谢 Longfei Yun、Shravan Konduru、Hao-Ting Tso、ChujiaGuo、Yu-shu Chen、Chace Zhu、Shivam Pansuria 和 Katherine Guo 在本工作初期阶段的贡献。本材料基于 AWS、谷歌和 Meta 的资助支持。本文中表达的任何观点、发现、结论或建议均为作者本人观点，不一定反映这些机构的立场。

影响声明

本文介绍的工作旨在推动机器学习领域的发展。我们的工作可能带来诸多潜在的社会影响，但我们认为无需在此逐一强调。

参考文献

Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., 等人。GPT-4 技术报告. *arXiv* 预印本 *arXiv:2303.08774*, 2023年。

Agrawal, A., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B. S., and Ramjee, R. Sarathi: 通过分块预填充与解码并行实现高效大语言模型推理. *arXiv*预印本 *arXiv:2308.16369*, 2023年8月。

AI, S. Bark：文本转语音模型. <https://github.com/suno-ai/bark>, 2023年。

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebron, F., and Sanghai, S. GQA：从多头检查点训练广义多查询变换器模型。见 *2023年自然语言处理经验方法会议论文集（EMNLP 2023）*，新加坡，2023年12月。

Aminabadi, R. Y., Rajbhandari, S., Awan, A. A., Li, C., Li, D., Zheng, E., Ruwase, O., Smith, S., Zhang, M., Rasley, J., 等。Deepspeed推理：以前所未有的规模实现高效的变换器模型推理。见*SC22：高性能计算、网络、存储与分析国际会议，德克萨斯州达拉斯*，2022年11月。IEEE。

Baeza-Yates, R., Ribeiro-Neto, B., 等。现代信息检索，第463卷。ACM出版社，纽约，1999。

Betker, J., Goh, G., Jing,L., Brooks, T., Wang, J., Li, L., Ouyang, L., Zhuang, J., Lee, J., Guo, Y., Man-

assra, W., Dhariwal, P., Chu, C., Jiao, Y., and Ramesh, A. 通过更好的描述改进图像生成. <https://cdn.openai.com/papers/dall-e-3.pdf>, 2024年。

Brockman, G., Eleti, A., Georges, E., Jang, J., Kilpatrick, L., Lim, R., Miller, L., and Pokrass, M. 介绍ChatGPT和Whisper APIs. <https://openai.com/blog/introducing-chatgpt-and-whisper-apis>, 2023年3月1日。Brysbaert, M. 我们每分钟阅读多少单词？阅读速度的回顾与荟萃分析。记忆与语言杂志，109:104047, 2019年。

Chase,H. LangChain. <https://github.com/langchain-ai/langchain>, 2022年10月。

Chen, L., Chen, Z., Tan, B., Long, S., Gasic, M., 和 Yu, K. Agentgraph: 面向通用对话管理的结构化深度强化学习方法. *arXiv*预印本 *arXiv:1905.11259*, 2019年5月。

Costa-juss`a, M. R., Cross, J., Celebi, O., Elbayad, M., Heafield, K., Heffernan, K., Kalbassi, E., Lam, J., Licht, D., Maillard, J., Sun, A., Wang, S., Wenzek, G., Young- blood, A., Akula, B., Barrault, L., Gonzalez, G. M., Hansanti, P., Hoffman, J., Jarrett, S., Sadagopan, K. R., Rowe, D., Spruit, S., Tran, C., Andrews, P., Ayan, N. F., Bhosale, S., Edunov, S., Fan, A., Gao, C., Goswami, V., Guzm`an, F., Koehn, P., Mourachko, A., Ropers, C., Saleem, S., Schwenk, H., and Wang, J. 不让任何语言掉队：扩展以人为本的机器翻译. *arXiv*预印本 *arXiv:2207.04672*, 2022年7月。

Greyling, C. 当使用ChatGPT API时，用户需要管理上下文. <https://cobusgreyling.medium.com/当-使用-ChatGPT-API-用户-将-不得不-管理-上下文-ba5869238913>, 2023年3月6日。

Gu, J., Stefani, E., Wu, Q., Thomason, J., 和 Wang, X. Vision-and-language navigation: 任务、方法及未来方向综述. 在*第60届计算语言学协会年会论文集（第一卷：长论文）*，都柏林，爱尔兰，2022. 计算语言学协会. doi: 10.18653/v1/2022.acl-long.524.

Hao, S., Liu, T., Wang, Z., 和 Hu, Z. Toolkengpt: 通过工具嵌入增强冻结语言模型与大量工具. 在*Advances in Neural InformationProcessingSystems36*, 美国路易斯安那州新奥尔良，2023年12月。

Hu, C., Huang, H., Xu, L., Chen, X., Xu, J., Chen, S., Feng, H., Wang, C., Wang, S., Bao, Y., Sun, N., and Shan, Y. Inference without interference: Disaggregate llm inference for mixed downstream workloads. *arXivpreprint arXiv:2401.11181*, 2024年1月。

INFERCEPT: Efficient Intercept Support for Augmented LLM Inference

Hu, Y., Lin, F., Zhang, T., Yi, L., and Gao, Y. Look before you leap: Unveiling the power of gpt-4v in robotic vision-language planning. *arXiv preprint arXiv:2311.17842*, November 2023.

Huang, W., Abbeel, P., Pathak, D., and Mordatch, I. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *Proceedings of 39th International Conference on Machine Learning*, Honolulu, Hawai’i, 2022.

Izacard, G., Lewis, P., Lomeli, M., Hosseini, L., Petroni, F., Schick, T., Dwivedi-Yu, J., Joulin, A., Riedel, S., and Grave, E. Atlas: Few-shot learning with retrieval augmented language models, 2022.

Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., A, S. V., Haq, S., Sharma, A., Joshi, T. T., Moazam, H., Miller, H., Zaharia, M., and Potts, C. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *The Twelfth International Conference on Learning Representations (ICLR ’24)*, Vienna, Austria, 2024. URL <https://openreview.net/forum?id=sY5N0zY5Od>.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, Koblenz, Germany, October 2023.

Li, Z., Zheng, L., Zhong, Y., Liu, V., Sheng, Y., Jin, X., Huang, Y., Chen, Z., Zhang, H., Gonzalez, J. E., and Stoica, I. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, Boston, MA, July 2023.

Lu, P., Peng, B., Cheng, H., Galley, M., Chang, K.-W., Wu, Y. N., Zhu, S.-C., and Gao, J. Chameleon: Plug-and-play compositional reasoning with large language models. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (NeurIPS ’23)*, New Orleans, Louisiana, December 2023.

Ma, Z., Edge, D., Findlater, L., and Tan, H. Z. Haptic keyclick feedback improves typing speed and reduces typing errors on a flat keyboard. In *2015 IEEE World Haptics Conference (WHC)*, Evanston, Illinois, 2015. IEEE.

Meta. Meta llama 3. <https://llama.meta.com/llama3/>, 2024.

Mialon, G., Dessi, R., Lomeli, M., Nalmpantis, C., Pasunuru, R., Raileanu, R., Roziere, B., Schick, T., Dwivedi-Yu, J., Celikyilmaz, A., Grave, E., LeCun, Y., and Scialom, T. Augmented language models: a survey. *Transactions on Machine Learning Research (TMLR)*, 2023. ISSN 2835-8856. URL <https://openreview.net/forum?id=jh7wH2AzKK>. Survey Certification.

Ng, A. The batch weekly issues 241. <https://www.deeplearning.ai/the-batch/issue-241/>, March 2024.

NVIDIA. Fastertransformer. <https://github.com/NVIDIA/FasterTransformer>, 2023.

OpenAI. ChatGPT plugins. <https://openai.com/blog/chatgpt-plugins>, March 2023.

OpenTable. New: Chatgpt restaurant recs, powered by opentable. <https://www.opentable.com/blog/chatgpt/>, March 23 2023.

Parisi, A., Zhao, Y., and Fiedel, N. Talm: Tool augmented language models. *arXiv preprint arXiv:2205.12255*, May 2022.

Patel, P., Choukse, E., Zhang, C., Shah, A., Goiri, Í., Maleki, S., and Bianchini, R. Splitwise: Efficient generative llm inference using phase splitting. In *The 53th International Symposium on Computer Architecture (ISCA 2024)*, Buenos Aires, Argentina, June 2024.

Patil, S. G., Zhang, T., Wang, X., and Gonzalez, J. E. Gorilla: Large language model connected with massive apis. *arXiv preprint arXiv:2305.15334*, 2023.

Qian, C., Han, C., Fung, Y., Qin, Y., Liu, Z., and Ji, H. CREATOR: Tool creation for disentangling abstract and concrete reasoning of large language models. In Bouamor, H., Pino, J., and Bali, K. (eds.), *Findings of the Association for Computational Linguistics: (EMNLP ’23)*, Singapore, December 2023. URL <https://aclanthology.org/2023.findings-emnlp.462>.

Qin, Y., Hu, S., Lin, Y., Chen, W., Ding, N., Cui, G., Zeng, Z., Huang, Y., Xiao, C., Han, C., Fung, Y. R., Su, Y., Wang, H., Qian, C., Tian, R., Zhu, K., Liang, S., Shen, X., Xu, B., Zhang, Z., Ye, Y., Li, B., Tang, Z., Yi, J., Zhu, Y., Dai, Z., Yan, L., Cong, X., Lu, Y., Zhao, W., Huang, Y., Yan, J., Han, X., Sun, X., Li, D., Phang, J., Yang, C., Wu, T., Ji, H., Liu, Z., and Sun, M. Tool learning with foundation models. *arXiv preprint arXiv:2304.08354*, June 2023.

Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. High-resolution image synthesis with latent diffusion models. *arXiv preprint arXiv:2112.10752*, 2021.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., and

InferCept: 用于增强LLM推理的高效拦截支持

Hu, Y., Lin, F., Zhang, T., Yi, L., and Gao, Y. Look before you leap: Unveiling the power of gpt-4v in robotic vision-language planning. *arXiv preprint arXiv:2311.17842*, 2023年11月.

Huang, W., Abbeel, P., Pathak, D., and Mordatch, I. 语言模型作为零样本规划器：为具身智能体提取可操作性知识。在 第39届国际机器学习大会论文集，夏威夷檀香山，2022年。

Izacard, G., Lewis, P., Lomeli, M., Hosseini, L., Petroni, F., Schick, T., Dwivedi-Yu, J., Joulin, A., Riedel, S., and Grave, E. Atlas：基于检索增强语言模型的少样本学习，2022年。

Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., A, S. V., Haq, S., Sharma, A., Joshi, T. T., Moazam, H., Miller, H., Zaharia, M., and Potts, C. DSPy：将声明式语言模型调用编译成最先进的流水线。在 第十二届国际学习表征会议（ICLR ’24），奥地利维也纳，2024年。网址<https://openreview.net/forum?id=sY5N0zY5Od>.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. 基于PagedAttention的大语言模型服务高效内存管理。见第29届操作系统原理研讨会论文集，德国科布伦茨，2023年10月。

Li, Z., Zheng, L., Zhong, Y., Liu, V., Sheng, Y., Jin, X., Huang, Y., Chen, Z., Zhang, H., Gonzalez, J. E., and Stoica, I. AlpaServe：面向深度学习服务的模型并行统计复用。见第17届USENIX操作系统设计与实现研讨会（OSDI 23），美国马萨诸塞州波士顿，2023年7月。

Lu, P., Peng, B., Cheng, H., Galley, M., Chang, K.-W., Wu, Y. N., Zhu, S.-C., and Gao, J. Chameleon：基于大语言模型的即插即用组合推理。见第37届国际神经信息处理系统会议（NeurIPS ’23）论文集，美国路易斯安那州新奥尔良，2023年12月。

Ma, Z., Edge, D., Findlater, L., 和 Tan, H. Z. 触觉按键反馈提高了平板键盘上的打字速度并减少了打字错误。在2015年IEEE世界触觉会议（WHC），伊利诺伊州埃文斯顿，2015年。IEEE.

Meta. Meta Llama 3. <https://llama.meta.com/llama3/>, 2024.

Mialon, G., Dessi, R., Lomeli, M., Nalmpantis, C., Pasunuru, R., Raileanu, R., Roziere, B., Schick, T., Dwivedi-Yu, J., Celikyilmaz, A., Grave, E., LeCun, Y., and Scialom, T. 增强语言模型：一项综述。机器学习研究汇刊(TMLR), 2023年. ISSN 2835-8856. 网址<https://openreview.net/forum?id=jh7wH2AzKK>. 调查认证. Ng, A. The Batch周刊第241期. <https://www.deeplearning.ai/the-batch/issue-241/>, 2024年3月.

英伟达. FasterTransformer. <https://github.com/NVIDIA/FasterTransformer>, 2023.

OpenAI. ChatGPT插件. <https://openai.com/blog/chatgpt-plugins>,2023年3月.

OpenTable. 新：Chatgpt餐厅推荐，由OpenTable提供支持. <https://www.opentable.com/blog/chatgpt/>, 2023年3月23日.

Parisi, A., Zhao, Y., 和 Fiedel, N. Talm：工具增强语言模型. *arXiv*预印本 *arXiv:2205.12255*, 2022年5月.

Patel, P., Choukse, E., Zhang, C., Shah, A., Goiri, I., Maleki, S., and Bianchini, R. Splitwise: 基于阶段分割的高效生成式大语言模型推理。发表于第53届国际计算机体系结构研讨会（ISCA 2024），阿根廷布宜诺斯艾利斯，2024年6月。

Patil, S. G., Zhang, T., Wang, X., and Gonzalez, J. E. Gorilla: 连接海量API的大型语言模型. *arXiv*预印本 *arXiv:2305.15334*, 2023年。

Qian, C., Han, C., Fung, Y., Qin, Y., Liu, Z., and Ji, H. CREATOR: 为解耦大型语言模型的抽象与具体推理而进行的工具创建。载于Bouamor, H., Pino, J., and Bali, K.（编著），《计算语言学协会发现：（EMNLP ’23）》，新加坡，2023年12月。URL<https://aclanthology.org/2023.findings-emnlp.462>.

Qin, Y., Hu, S., Lin, Y., Chen, W., Ding, N., Cui, G., Zeng, Z., Huang, Y., Xiao, C., Han, C., Fung, Y. R., Su, Y., Wang, H., Qian, C., Tian, R., Zhu, K., Liang, S., Shen, X., Xu, B.,Zhang, Z., Ye, Y., Li, B., Tang, Z., Yi, J., Zhu, Y., Dai, Z., Yan, L., Cong, X., Lu, Y., Zhao, W., Huang, Y., Yan, J., Han, X., Sun, X., Li, D., Phang, J., Yang, C., Wu, T., Ji, H., Liu, Z., and Sun, M. 基于基础模型的工具学习. *arXiv*预印本 *arXiv:2304.08354*, 2023年6月。

Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. 基于潜在扩散模型的高分辨率图像合成. *arXiv*预印本 *arXiv:2112.10752*, 2021年。

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., and

Scialom, T. Toolformer: Language models can teach themselves to use tools. *37th Conference on Neural Information Processing Systems, 2023*.

Shen, Y., Song, K., Tan, X., Li, D., Lu, W., and Zhuang, Y. HuggingGPT: Solving AI tasks with chatGPT and its friends in hugging face. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (NeurIPS ’23)*, New Orleans, Louisiana, December 2023.

Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., and Catanzaro, B. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

Shridhar, M., Yuan, X., Côté, M.-A., Bisk, Y., Trischler, A., and Hausknecht, M. Alfworld: Aligning text and embodied environments for interactive learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, Virtual, 2021.

Srivatsa, V., He, Z., Abhyankar, R., Li, D., and Zhang, Y. Preble: Efficient distributed prompt scheduling for llm serving. *UCSD CSE Technical Reports*, May 2024. URL <https://escholarship.org/uc/item/1bm0k1w0>.

Suris, D., Menon, S., and Vondrick, C. Vipergpt: Visual inference via python execution for reasoning. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV ’23)*, pp. 11854–11864, Los Alamitos, CA, USA, October 2023. URL <https://doi.ieeecomputersociety.org/10.1109/ICCV51070.2023.01092>.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Vaidya, N., Comly, N., DeLaere, J., Patel, A., and Oh, F. Nvidia tensorrt-llm supercharges large language model inference on nvidia h100 gpus. <https://developer.nvidia.com/blog/nvidia-tensorrt-llm-supercharges-large-language-model-inference-on-nvidia-h100-gpus/>, 2023.

Wang, B. and Komatsuzaki, A. GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model. <https://github.com/kingoflolz/mesh-transformer-jax>, May 2021.

Wang, L., Ma, C., Feng, X., Zhang, Z., ran Yang, H., Zhang, J., Chen, Z.-Y., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., and rong Wen, J. A survey on large language model based autonomous agents. *arXiv*

preprint arXiv:2308.11432, August 2023. URL <https://api.semanticscholar.org/CorpusID:261064713>.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, New Orleans, Louisiana, 2023.

Wolfram, S. Chatgpt gets its ‘wolfram superpowers’! <https://writings.stephenwolfram.com/2023/03/chatgpt-gets-its-wolfram-superpowers/>, March 2023.

Wu, B., Zhong, Y., Zhang, Z., Huang, G., Liu, X., and Jin, X. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920*, May 2023.

Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., and Manning, C. D. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600*, 2018.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, Kigali, Rwanda, May 2023.

Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., and Chun, B.-G. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI ’22)*, Carlsbad, CA, July 2022.

Zhang, Z., Zhang, A., Li, M., and Smola, A. Automatic chain of thought prompting in large language models. In *The Eleventh International Conference on Learning Representations*, Kigali, Rwanda, May 2023.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., Zhang, H., Gonzalez, J. E., and Stoica, I. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, New Orleans, Louisiana, December 2023a.

Zheng, L., Yin, L., Xie, Z., Huang, J., Sun, C., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. Efficiently programming large language models using sglang. *arXiv preprint arXiv:2312.07104*, December 2023b.

Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., and Zhang, H. Distllm: Disaggregating prefill and decoding for goodput-optimized large language model

Scialom, T. Toolformer: Language models can teach

自身使用工具。第37届神经网络信息处理系统大会，2023年。Shen, Y., Song, K., Tan, X., Li, D., Lu, W., and Zhuang, Y. HuggingGPT: 使用ChatGPT及其Hugging Face上的同伴解决AI任务。见第37届国际神经信息处理系统会议论文集 (NeurIPS ’23), 美国路易斯安那州新奥尔良，2023年12月。Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., and Catanzaro, B. Megatron-lm: 使用模型并行训练数十亿参数的语言模型。arXiv预印本 arXiv:1909.08053, 2019年。Shridhar, M., Yuan, X., Côté, M.-A., Bisk, Y., Trischler, A., and Hausknecht, M. Alfworld: 对齐文本与具身环境以实现交互式学习。见国际学习表征会议论文集 (ICLR), 线上会议，2021年。Srivatsa, V., He, Z., Abhyankar, R., Li, D., and Zhang, Y. Preble: 用于大语言模型服务的高效分布式提示调度。UCSD CSE技术报告，2024年5月。URL <https://escholarship.org/uc/item/1bm0k1w0>. Suris, D., Menon, S., and Vondrick, C. Vipergpt: 通过Python执行进行视觉推理以进行推理。见2023年 IEEE/CVF国际计算机视觉会议 (ICCV ’23), 第11854–11864页，美国加州洛杉矶拉米托斯，2023年10月。URL <https://doi.ieeecomputersociety.org/10.1109/ICCV51070.2023.01092>. Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: 开放且高效的基础语言模型。arXiv预印本 arXiv:2302.13971, 2023年。Vaidya, N., Comly, N., DeLaere, J., Patel, A., and Oh, F. 英伟达TensorRT-LLM极大地加速了英伟达H100 GPU上的大语言模型推理。

h
t
t
p
s://
d
e
v
e
l
o
p
e
r.
n
v
i
d
i
a.
c
o
m/
b
l
o
g/
n
v
i
d
i
a-
t
e
n
s
o
r
t-ll
m-supercharges-large-language-model-inference-on-nvidia-h100-gpus/, 2023年。Wang, B. and Komatsuzaki, A. GPT-J-6B: 一个60亿参数的自回归语言模型。 <https://github.com/kingoflolz/mesh-transformer-jax>, 2021年5月。Wang, L., Ma, C., Feng, X., Zhang, Z., ran Yang, H., Zhang, J., Chen, Z.-Y., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., and rong Wen, J. A survey on lar

ge language 模型 基于 自主 agentic 实体。 *arXiv*

预印本 *arXiv:2308.11432*, 2023年8月。 URL<https://api.semanticscholar.org/CorpusID:261064713>. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., and Zhou, D. 思维链提示在大语言模型中激发推理。在第36届国际神经信息处理系统会议论文集，美国路易斯安那州新奥尔良，2023年。Wolfram, S. Chatgpt 获得其 ‘wolfram 超级能力’ ！

h
t
t
p
s://
w
r
i
t
i
n
g
s.
s
t
e
p
h
e
nwolfram.com/2023/03/chatgpt-gets-its-wolfram-superpowers/, 2023年3月。Wu, B., Zhong, Y., Zhang, Z., Huang, G., Liu, X., and Jin, X. 大语言模型的快速分布式推理服务。 *arXiv*预印本 *arXiv:2305.05920*, 2023年5月。Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., and Manning, C. D. HotpotQA: 一个用于多样化、可解释的多跳问答的数据集。 *arXiv*预印本 *arXiv:1809.09600*, 2018年。Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. ReAct: 语言模型中推理与行动的协同。在第十一届国际学习表征会议，卢旺达基加利，2023年5月。Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., and Chun, B.-G. Orca: 基于Transformer的生成模型的分布式服务系统。在第16届USENIX操作系统设计与实现研讨会（OSDI ’22），加利福尼亚州卡尔斯巴德，2022年7月。Zhang, Z., Zhang, A., Li, M., and Smola, A. 大语言模型中的自动思维链提示。在第十一届国际学习表征会议，卢旺达基加利，2023年5月。Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., Zhang, H., Gonzalez, J. E., and Stoica, I. 使用MT-bench和聊天机器人竞技场评判LLM作为评判者。在第三十七届神经网络信息处理系统会议数据集与基准轨迹，美国路易斯安那州新奥尔良，2023年12月a。Zheng, L., Yin, L., Xie, Z., Huang, J., Sun, C., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. 使用SGLang高效编程大语言模型。 *arXiv*预印本 *arXiv:2312.07104*, 2023年12月b。Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., and Zhang, H. DistLLM: 分离预填充和解码以实现好吞吐率优化的大语言模型

serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*, Santa Clara, CA, July 2024.

服务。在第18届USENIX操作系统设计与实现研讨会（OSDI '24）论文集，加利福尼亚州圣克拉拉，2024年7月。

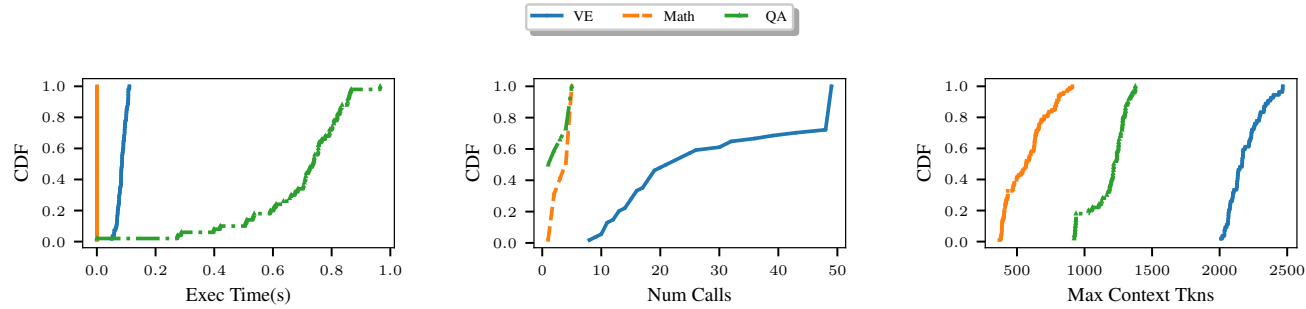


Figure 4: **CDF Results of Short APIs.** Each line plots the CDF distribution of all the calls of one API type.

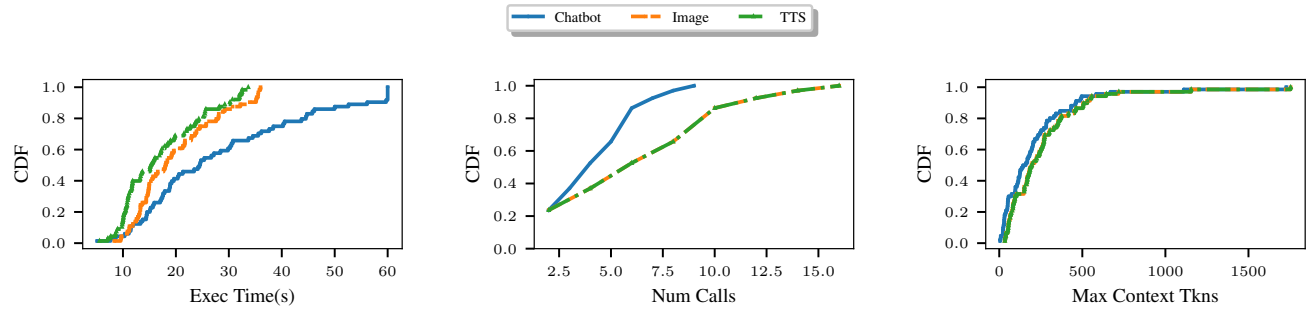


Figure 5: **CDF Results of Long APIs.** Each line plots the CDF distribution of all the calls of one API type.

API Properties and Dataset Generation

We analyze the properties of different datasets to evaluate the effectiveness of our inference system. We consider datasets from arithmetic, a knowledge-based question and answering system, a multi-step chat bot, and an embodied virtual environment. The dataset demonstrates different execution time, frequency of API calls, and context lengths. We augment the datasets with relevant APIs from calculator, wikipedia, and a virtual environment gym.

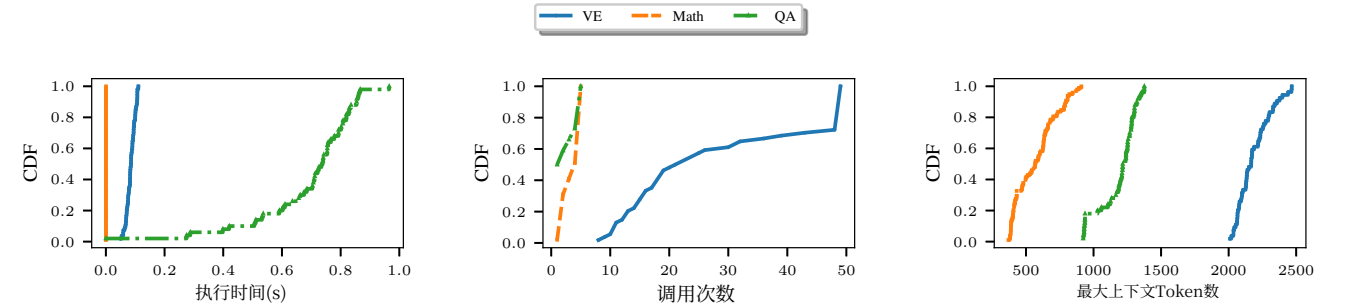
We measure the API execution time, the number of API calls, the number of returned tokens of an API call, and the context length when an API is called. Figures 4 and 5 present the detailed CDF results the four metrics for short-running APIs (math, QA, VE) and long-running APIs (chatbot, image, TTS). Below, we briefly introduce each API type and discuss their properties.

We use the ReAct framework to prompt the LLM to call a Wikipedia endpoint with different questions. The responses from the Wikipedia endpoint are postprocessed. We use the live Wikipedia API endpoint for our benchmark and For each call, we retrieve a summary of the relevant Wikipedia page, which we post-process by limiting the size of the responses to fit within the maximum model sequence length.

To evaluate VE, we prompt GPT-4 LLM with an initial virtual environment and use the ReAct event loop to repeatedly interact with the environment. For our evaluation, we truncate to fit within the context length.

We prompt GPT-4 to generate complex Stable Diffusion prompts at a certain size. Each prompt will trigger an API call of the Stable Diffusion model (Rombach et al., 2021) to generate an image. Each round with the LLM involves a user prompt with normal decoding and then a diffusion call. The number of calls total is modeled around the chat dataset. We estimate the number of image generation rounds (*i.e.*, number of API calls) to be the same as the number of chatting rounds in Chatbot. As there is no standard way of returning tokens from an image generation, we use a short, constant-length sentence describing the generated image as returned tokens. For API execution time, we measure the actual Stable Diffusion call and estimate the human response time in the same as as Chatbot; the sum of them at each round is shown in Table 1 and Figure 5. For the context and return length, we provide a mixture of ShareGPT dataset (Zheng et al., 2023a) and the real prompts.

We use a methodology similar to image generation to understand TTS-augmented LLMs.



Fi图4：短API的CDF结果。每条线绘制了某个API类型所有调用的CDF分布

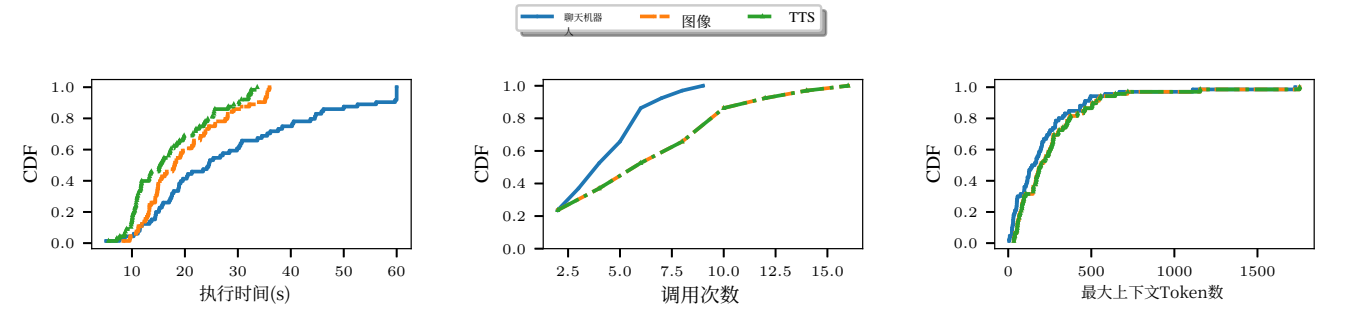


图5：长API的CDF结果。每条线绘制了某个API类型所有调用的CDF分布。

API属性与数据集生成

我们分析不同数据集的属性，以评估推理系统的有效性。我们考虑了来自算术、基于知识的问答系统、多步聊天机器人和具身虚拟环境的数据集。该数据集展示了不同的执行时间、API调用频率和上下文长度。我们通过计算器、维基百科和虚拟环境健身房的相关API对数据集进行了增强。

我们测量了API执行时间、API调用次数、每次API调用返回的令牌数以及调用API时的上下文长度。图4和5展示了短运行API（数学、问答、虚拟环境）和长运行API（聊天机器人、图像、文本转语音）在四个指标上的详细累积分布函数结果。下面，我们简要介绍每种API类型并讨论其特性。

我们使用ReAct框架提示大语言模型调用维基百科端点来回答不同的问题。来自维基百科端点的响应会经过后处理。我们使用实时维基百科API端点进行基准测试。对于每次调用，我们检索相关维基百科页面的摘要，并通过限制响应大小以适配最大模型序列长度进行后处理。

为了评估虚拟环境，我们使用初始虚拟环境提示GPT-4大语言模型，并使用ReAct事件循环反复与环境交互。为了我们的评估，我们进行截断以适应上下文长度。

我们提示GPT-4生成特定大小的复杂Stable Diffusion提示。每个提示将触发一次Stable Diffusion模型（Rombach等人, 2021年）生成一张图像。每一轮与大语言模型的交互包括一个带有正常解码的用户提示，然后是一个扩散调用。总调用次数基于聊天数据集建模。我们估计图像生成轮数（即，API调用次数）与聊天机器人中的聊天轮数相同。由于没有从图像生成返回标记的标准方式，我们使用一个描述生成图像的短定长句子作为返回标记。对于API执行时间，我们测量实际的Stable Diffusion调用，并以与聊天机器人相同的方式估计人类响应时间；每一轮的总和如表1和图5。对于上下文和返回长度，我们提供ShareGPT数据集（Zheng et al., 2023a）与真实提示。

我们采用类似于图像生成的方法来理解TTS增强的大语言模型。

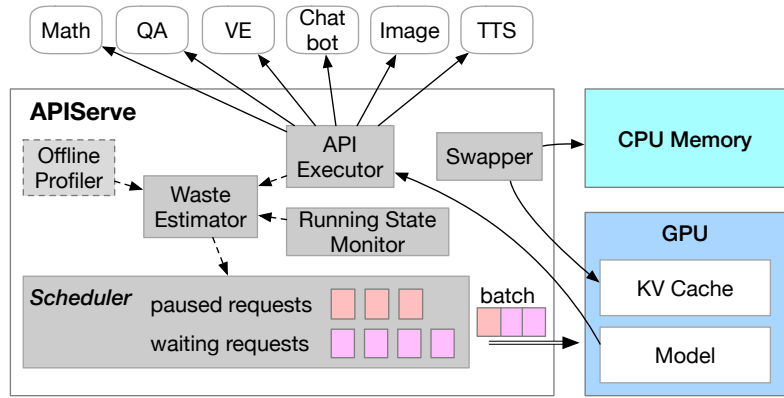


Figure 6: Overall INFERCEPT Architecture.

Implementation Details

We implement INFERCEPT on top of vLLM and build INFERCEPT, an inference system designed for API-augmented LLMs. As shown in Figure 6, INFERCEPT includes an API executor that performs different API calls, an iterative-level scheduler that forms a batch of waiting requests and (partial) paused requests, a swap manager that facilitates GPU/CPU memory swapping, a running status monitor that monitors the resource usage and size of running requests, a waste estimator that calculates GPU memory waste for each API-paused request, and an offline profiler that collects basic systems metrics before starting serving.

At the end of each iteration, the scheduler gathers all requests that trigger API calls, adds them to the paused-request queue, swaps as much context (computed keys and values) of paused requests as allowed to CPU memory, determines whether to preserve or discard the remaining context of each paused request based on the GPU memory waste amount calculated by the waste estimator, and calls the actual API. When an API call finishes, INFERCEPT determines how many swapped-out or discarded tokens to swap in or recompute in the next iteration. It also determines how many waiting requests (non-API paused requests) to schedule in the next iteration. When all the context of a paused request has been restored, INFERCEPT adds the tokens returned by the API call to the resumed request.

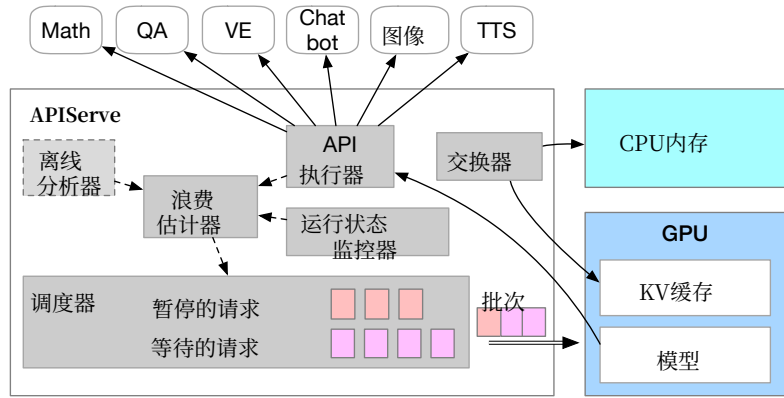


图6：整体 INFERCEPT 架构。

实现细节

我们实现 INFERCEPT 在vLLM之上并构建 INFERCEPT, 一个专为API增强的大语言模型设计的推理系统。如图 6 所示, INFERCEPT包括: 一个执行不同API调用的API执行器, 一个将等待的请求和（部分）暂停的请求组成批次的迭代级调度器, 一个促进GPU/CPU内存交换的交换管理器, 一个监控资源使用和正在运行的请求大小的运行状态监视器, 一个计算每个API暂停请求的GPU内存浪费的浪费估算器, 以及一个在开始服务前收集基本系统指标的离线分析器。

在每次迭代结束时, 调度器收集所有触发 API 调用的请求, 将其添加到暂停请求队列中, 将暂停请求的上下文（计算出的键值对）尽可能多地换入到 CPU 内存中, 根据浪费估算器计算的 GPU 内存浪费量决定每个暂停请求的剩余上下文是保留还是丢弃, 并调用实际 API。当 API 调用完成后, INFERCEPT 确定在下次迭代中需要换入或重新计算已换出或丢弃的令牌数量。它还确定在下次迭代中安排多少等待请求（非 API 暂停请求）。当暂停请求的所有上下文都已恢复后, INFERCEPT 将 API 调用返回的令牌添加到恢复的请求中。